

Lekcija 2 – Klase i objekti

Pregled

- 1.1 Osnovni pojmovi
- 1.2 Objekti
- 1.3 Implementacija i interfejs klase
- 1.4 Preprocesiranje
- 1.5 Pokazivači
- 1.6 Reference
- 1.7 Objekti i funkcije članice
- 1.8 Pokazivač *this*
- 1.9 Preklopljene funkcije
- 1.10 *Inline* funkcije
- 1.11 Konstante funkcije
- 1.12 Podrazumijevani argumenti
- 1.13 Konstruktori
- 1.14 Destruktori
- 1.15 Objekti kao arugmenti funkcija
- 1.16 Statički (dijeljeni) atributi
- 1.17 Statičke (dijeljene) metode
- 1.18 Prijatelji klasa

Ciljevi lekcije

- U ovoj lekciji:
 - Upoznavanje sa osnovim pojmovima vezanim za klase i objekte
 - Direktive preprocesora
 - Pokazivači i reference
 - Značenje pokazivača *this*
 - Preklapanje imena funkcija
 - *Inline* funkcije
 - Konstruktori i destruktori
 - Konstruktor kopije
 - Stački podaci članovi i statičke metode
 - Prijatelji klasa

1.1 Osnovni pojmovi

- Klasa predstavlja apstrakciju zajedničkih atributa i zajedničkog ponašanja jednog skupa srodnih entiteta
- Klase sadrže:
 - Podatke (definišu stanje) i
 - Funkcije (definišu ponašanje)
- Podaci koji su dio klase nazivaju se:
 - Podaci članovi, ili atributi
- Funkcije koje su dio klase nazivaju se:
 - Funkcije članice
 - Često se koristi naziv *metode*

1.1 Osnovni pojmovi

- Svaka klasa ima sekcije koje određuju pravo pristupa članovima klase
 - To se odnosi i na podatke i na metode
 - Princip enkapsulacije
- Sekcija može biti označena kao
 - Private
 - Nazivaju se privatnim članovicama klase
 - Ne može im se pristupiti “spolja”
 - Vidjivi su samo ostalim članicama klase
 - Public
 - Nazivaju se javnim članicama klase
 - Nema ograničenja u pogledu pristupa
 - Protected
 - Nazivaju se zaštićenim članicama klase
 - Vidjivi su ostalim članicama klase, kao i članicama izvedenih klasa
- Sekcije mogu biti navedene u proizvoljnom redosljedu
 - Može biti više sekcija sa istom oznakom

1.1 Osnovni pojmovi

```
class Complex //deklaracija klase
{
private:
    float re; //podaci clanovi
    float im;

public:
    void setData(float r, float i) //metoda
    {
        re = r;
        im = i;
    }

    void Print() //metoda
    {cout << "Re " << re << " Im " << im << endl;}
};
```

1.1 Osnovni pojmovi

```
int main()
{
    Complex c1, c2; //kreiranje objekata c1 i c2
    c1.setData(4.5, 2.3); //poziv metoda
    c2.setData(1.2, 0.5);

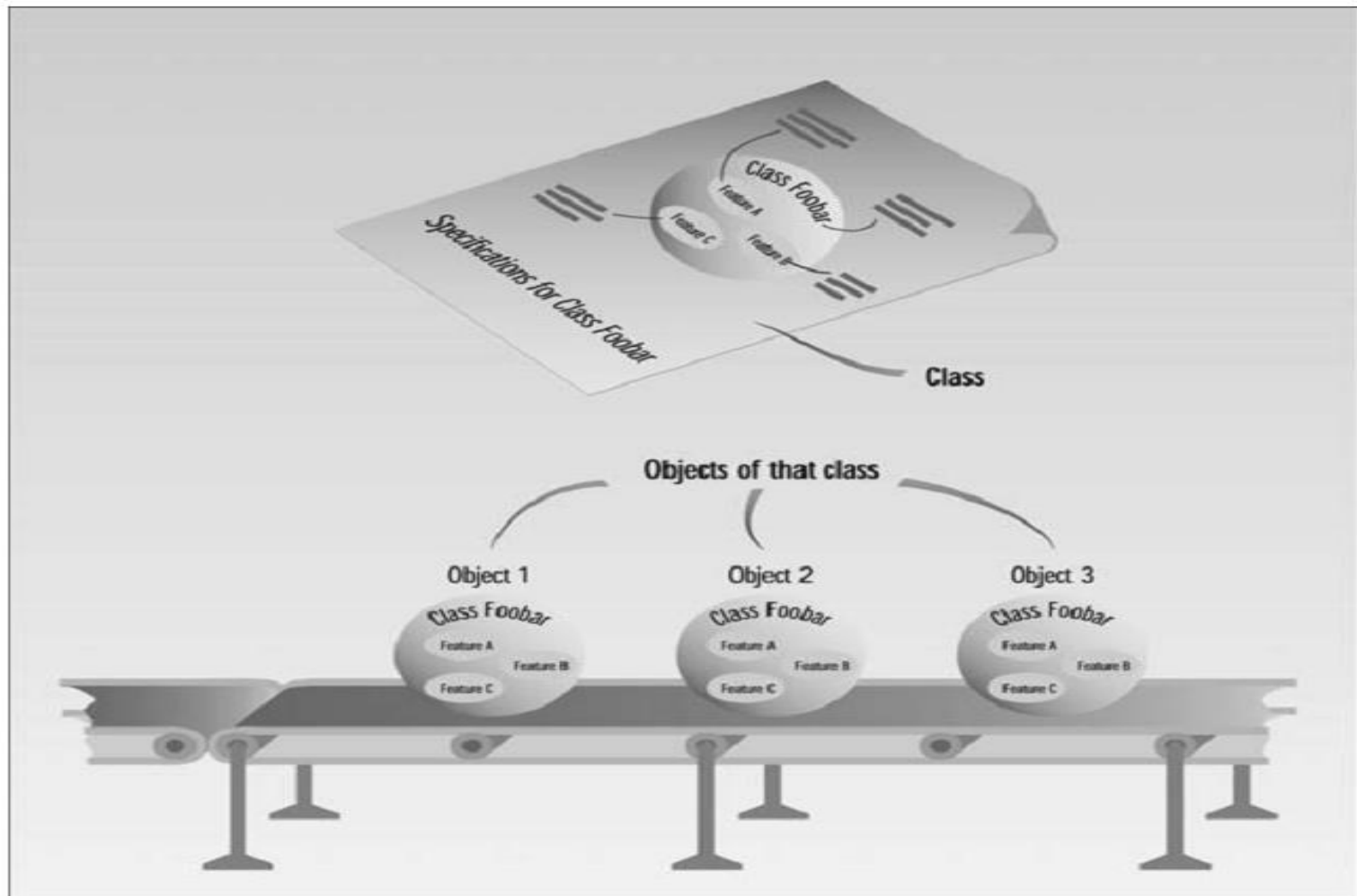
    c1.Print(); //poziv metoda za prikaz podataka
    c2.Print();
    return 0;
}
```

```
Re 4.5 Im 2.3
Re 1.2 Im 0.5
```

```
Process returned 0 (0x0)   execution time : 0.109 s
```

1.2 Objekti

- Jedna pojava (instanca) klase naziva se objekat

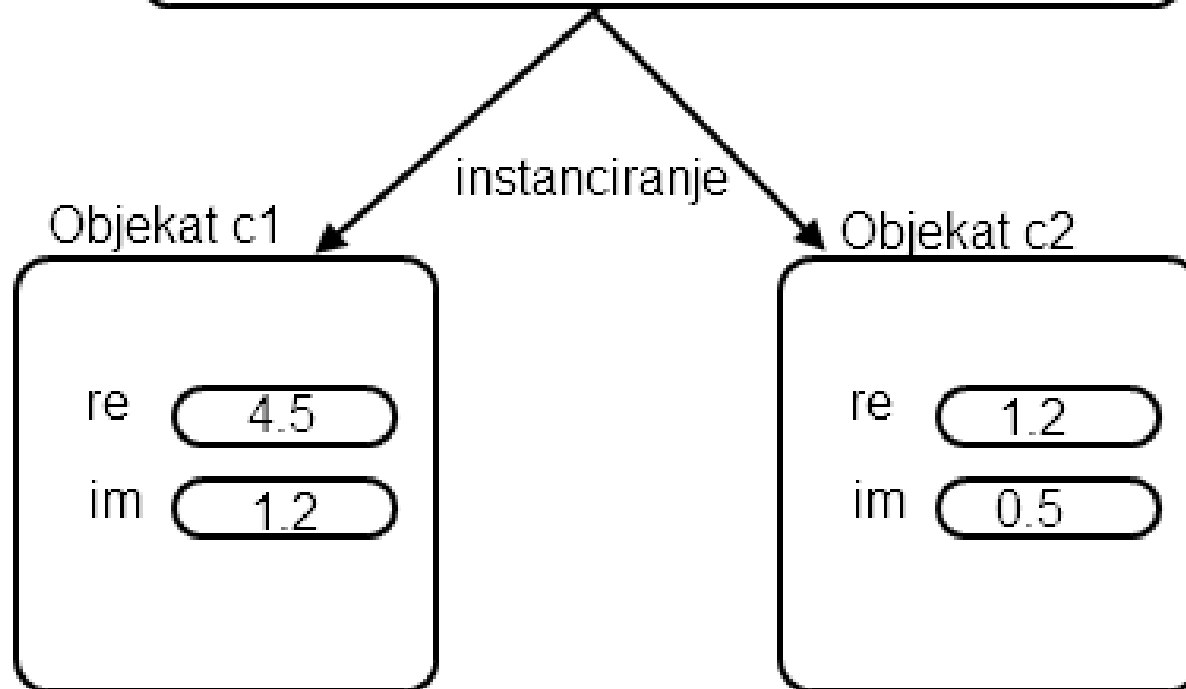


1.2 Objekti

- Objekti klase definišu (deklarišu) se na uobičajen način, kao i promjenjive osnovnih tipova
 - Naziv klase se koristi kao ime tipa, npr. `Complex C1;`
- Za svaki objekat klase formira se poseban komplet svih nestatičkih atributa te klase
 - Svaki objekat ima sopstvenu memoriju i u toj memoriji se čuvaju vrijednosti atributa (promjenjivih)
 - Pojam statičkih atributa biće detaljnije opisani u ostatku lekcije
 - U memoriji objekta se čuvaju i adrese funkcija članica

Specifikacija klase

```
class Complex  
{  
  private:  
    float re; //podaci clanovi  
    float im;  
    .....  
}
```



1.2 Objekti

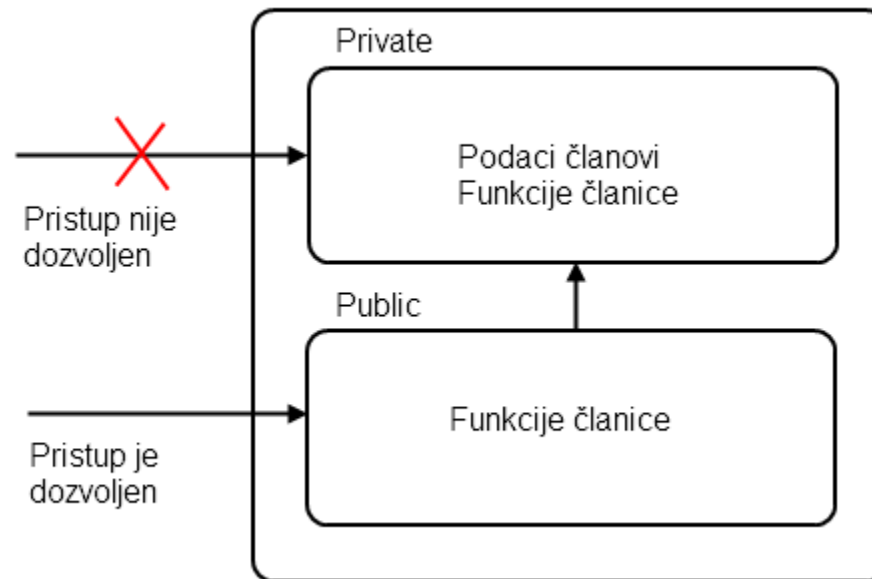
- Atributima i metodama pristupa se preko operatora tačka (.)
- Poziv metoda članica vrši se isključivo u kontekstu konkretnog objekta
 - Npr. `c1.Print()` ;
 - Poziv nema smisla vršiti u kontekstu cijele klase
 - Poziv `Complex.Print()` ; nema smisla i prevodilac bi prijavio grešku
- Poziv metoda se naziva *upućivanjem poruke* objektu klase
- Objekat koji šalje poruku (poziva metod) naziva se *objekat-klijent*
- Objekat koji prima poruku (čiji se metod poziva) je *objekat-server*
- Iz svoje metode objekat može pozivati metodu drugog objekta

1.2 Objekti

- Dozvoljeno je
 - Definisanje pokazivača na objekte
 - Definisanje nizova objekata
 - Dodeljivanje vrijednosti (operator =) jednog objekta drugom
 - Uzimanje adrese objekata (operator &)
 - Posredno pristupanje objektima preko pokazivača (operator *)
 - Neposredno pristupanje atributima i pozivanje metoda (operator .)
 - Posredno pristupanje atributima i metodama preko pokazivača (operator ->)
 - Pristupanje elementima nizova objekata (operator [])
 - Prenošnje objekata kao argumenata funkcija i to po vrijednosti ili referenci
 - Prenošnje pokazivača na objekte kao argumenata funkcija
 - Vraćanje objekata iz funkcija po vrijednosti ili referenci
 - Vraćanje pokazivača na objekte

1.2 Objekti

- Dobra programerska praksa je da svi podaci članovi budu privatni
 - Time se izbjegava neželjeni pristup podacima
 - Pristup podacima dozvoljen je jedino preko metoda članica



1.2 Objekti

```
class Time
{
    public:
        int hour;
        int min;
};

int main()
{
    Time t1;
    t1.hour = 25;
    //varijabla se dovodi u nevalidno stanje
    return 0;
}
```

1.2 Objekti

```
class Time
{
    private:
        int hour;
        int min;
    public:
        void setHour(int h)
        {
            if (h > 23 || h < 0)
            {
                cout << "Invalid hour" << endl;
                return;
            }
            hour = h;
        }
};
```

1.2 Objekti

```
int main()  
{  
    Time t1;  
    t1.setHour(25);  
    return 0;  
}
```

Invalid hour

Process returned 0 (0x0) execution time : 0.453 s

1.3 Implementacija i interfejs klase

- Svaka klasa ima implementaciju i interfejs
 - Implementaciju klase čine:
 - Privatni podaci članovi i
 - Definicije funkcija
 - Interfejs klase čine:
 - Javni podaci članovi (ne preporučuje se) i
 - Deklaracije funkcija
 - Praksa se da implementacija i interfejs razdvoje u posebne fajlove
 - Datoteka sa ekstenzijom *h* sadrži interfejs klase
 - Datoteka sa ekstenzijom *cpp* sadrži implementaciju klase

1.3 Implementacija i interfejs klase

```
class Complex
{
    private:
        float re; //deklaracija podataka
        float im;
    public:
        //zaglavlja funkcija
        void setData(float r, float i);
        void Print();
};
```

1.3 Implementacija i interfejs klase

```
#include "Complex.h"

void Complex::setData(float r, float i)
{
    re = r;
    im = i;
}

void Complex::Print()
{
    cout << "Re " << re << " Im " << im
    << endl;
}
```

1.3 Implementacija i interfejs klase

- Deklaracija funkcije članice obično se nalazi u interfejsu klase
 - Opšti oblik `T ime(lista_arg)`
 - `T` – povratni tip
 - `ime` – identifikator funkcije
 - `lista_arg` – lista formalnih argumenata
 - Lista je odvojena zarezima
 - Deklaracija argumenta ima oblik `T ime_arg`
- Definicija (implementacija) funkcije je zapravo deklaracija funkcije iza koje slijedi tijelo funkcije
 - Početak i kraj tijela funkcije označavaju `{ i }`
 - Povratna vrijednost funkcije jeste izraz naveden poslije ključne riječi `return`
 - Npr. `return 0;`

1.3 Implementacija i interfejs klase

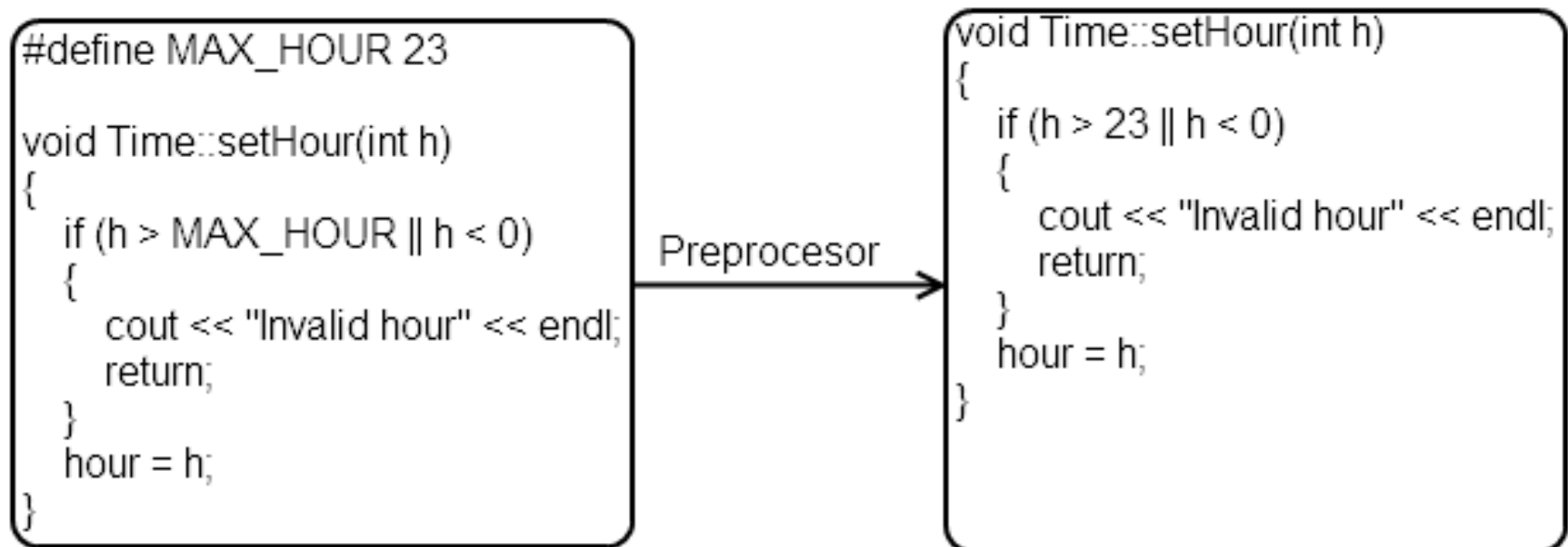
- Kod definicije funkcije prije imena navodi se ime klase zajedno sa operatorom ::
 - Npr. `void Complex::Print()`
- Operator ::
 - Operator razrešavanja oblasti važenja
 - Velika razlika između
 - definicije `void Complex::Print()`
 - Označava definiciju funkcije `Print` iz klase `Complex`
 - Definicija `void Print()`
 - Označava definiciju neke globalne funkcije `Print`

1.4 Preprocesiranje

- Nasljeđeno iz programskog jezika C
- Prije samog prevođenja izvorni kod programa se najprije procesira posebnim programom za obradu teksta
 - Ovaj program se naziva preprocesor
- Ulaz u proces prepocerisanja jeste izvorni kod, a rezultat je takođe izvorni kod
 - Procesor prevodiocu daje “očišćen” tekst programa
- Transformaciju koje treba obaviti nad tekstom zadaju se *direktivama*
 - Specijalne linije u programu koje počinju simbolom #

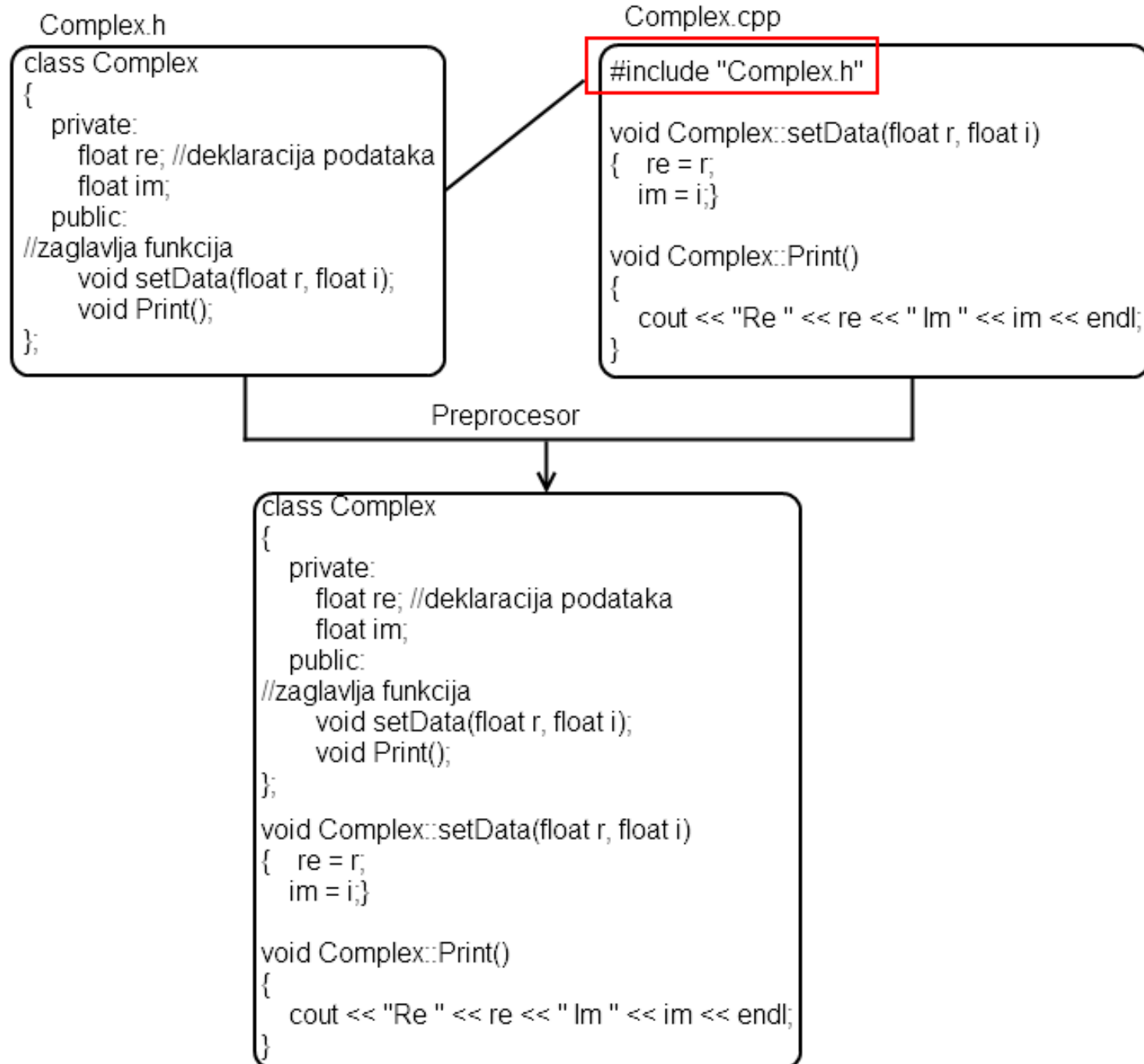
1.4 Preprocesiranje

- Direktiva `#define`
 - Definisane simboličkih konstanti
 - Opšti oblik `#define identifikator vrijednost`
 - Vršiti “slijepu” zamjenu svakog pojavljivanja identifikatora zadanom vrijednošću



1.4 Preprocesiranje

- Direktiva `#include`
 - Preprocesorska direktiva
 - Poziva se prije samog kompajliranja programa
 - Kopira kompletan sadržaj jednog fajla u drugi
 - Omogućava da se na jednostavan način učitava interfejs klase u više fajlova
 - Mogući oblici su
 - Direktiva `#include "imefajla"`
 - » Učitava korisnički definisan fajl sa tekućeg direktorijma
 - Direktiva `#include <imefajla>`
 - » Učitava sistemski (predefinisani) fajl sa odgovarajuće lokacije



1.4 Preprocesiranje

- Direktive `#ifdef`, `#ifndef` i `#endif`
 - Omogućavaju da se određeni dio koda isključi ili uključi u zavisnosti od toga da li je zadovoljen određeni uslov
 - Direktiva `#ifdef` dozvoljava kompajliranje dijela koda samo je definisana simbolička konstanta koja se javlja poslije ove direktive
 - Često se koristi kod fajlova koji sadrže zaglavlje neke klase, kako bi se izbjeglo višestruko učitavanje interfejsa klase

```
#define CHECK_HOUR 1
```

```
void Time::setHour(int h)
```

```
{
```

```
#ifndef CHECK_HOUR
```

```
    if (h > 23 || h < 0)
```

```
    {
```

```
        cout << "Invalid hour" << endl;
```

```
        return;
```

```
    }
```

```
    hour = h;
```

```
#else
```

```
    hour = h;
```

```
#endif
```

```
}
```

Preprocesor

```
void Time::setHour(int h)
```

```
{
```

```
    if (h > 23 || h < 0)
```

```
    {
```

```
        cout << "Invalid hour" << endl;
```

```
        return;
```

```
    }
```

```
    hour = h;
```

```
}
```

```
void Time::setHour(int h)
```

```
{
```

```
#ifndef CHECK_HOUR
```

```
    if (h > 23 || h < 0)
```

```
    {
```

```
        cout << "Invalid hour" << endl;
```

```
        return;
```

```
    }
```

```
    hour = h;
```

```
#else
```

```
    hour = h;
```

```
#endif
```

```
}
```

Preprocesor

```
void Time::setHour(int h)
```

```
{
```

```
    hour = h;
```

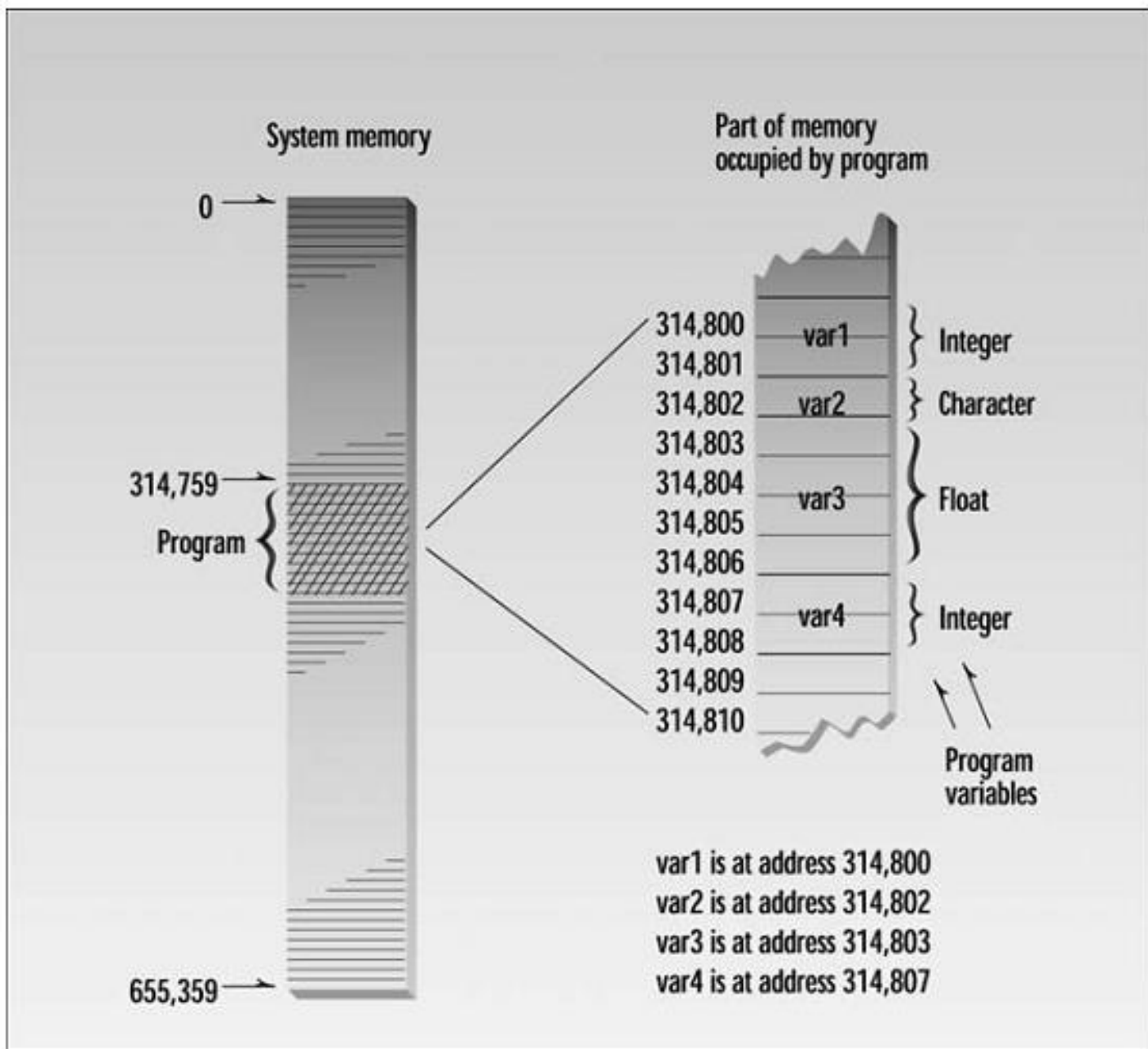
```
}
```

1.4 Preprocesiranje

```
#ifndef _COMPLEX_CLASS_  
#define _COMPLEX_CLASS_  
class Complex  
{  
    private:  
        float re;  
        float im;  
    public:  
        void setData(float r, float i);  
        void Print();  
};  
#endif
```

1.5 Pokazivači

- Za razliku od većine savremenih jezika (Java, C# itd.) jezik C++ podržava pokazivače
- Svaki bajt u memoriji ima svoju adresu
 - Adrese su redni brojevi
- Program koji je učitao u memoriju zauzima jedan opseg adresa
 - Svaka varijabla, kao i svaka funkcija ima svoju adresu
- Adresa neke promjenjive može se dobiti putem operatora &

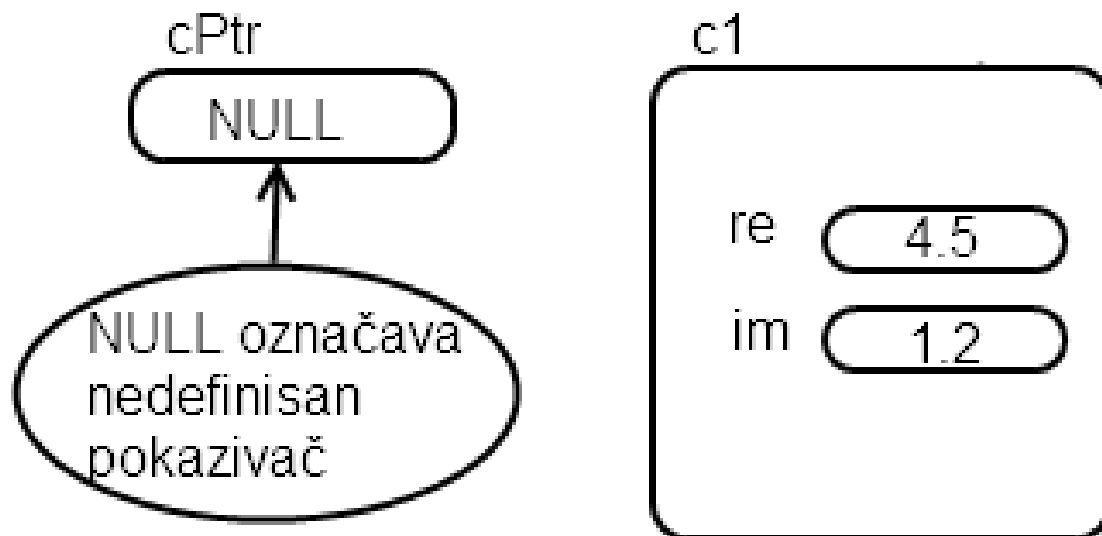


1.5 Pokazivači

- Moguće je deklarirati promjenljivu koja ne sadrži neku vrijednost već adresu
 - Pokazivačke varijable, tj. pokazivači
 - Pokazivačka varijabla deklarira se navođenjem * poslije naziva tipa podatka
 - Nema ograničenja u pogledu tipa podatka, tj. može se deklarirati pokazivač na bilo koji tip
- Deklaracija `Complex* cPtr` bitno je različita u odnosu na `Complex c1`
 - Promjenljiva `c1` predstavlja objekat, dok `cPtr` treba da sadrži adresu objekta klase `Complex`

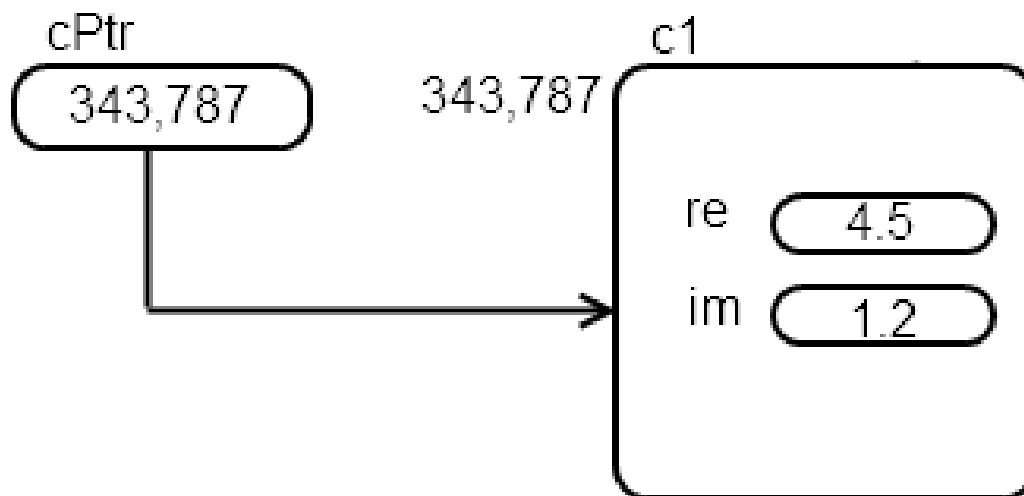
1.5 Pokazivači

```
Complex c1;  
c1.setData(4.5, 1.2);  
Complex* cPtr = NULL;
```



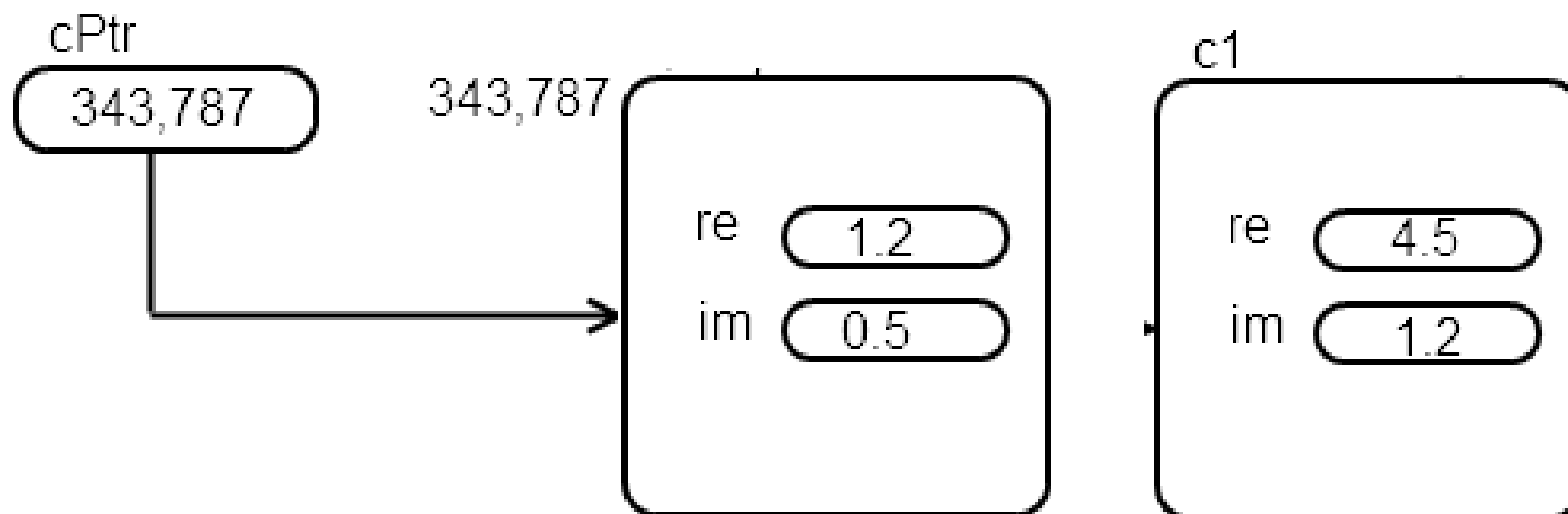
1.4 Pokazivači

```
Complex c1;  
c1.setData(4.5, 1.2);  
Complex* cPtr = &c1;
```



1.4 Pokazivači

```
Complex c1;  
c1.setData(4.5, 1.2);  
Complex* cPtr = new Complex;  
cPtr->setData(1.2, 0.5);
```



1.5 Pokazivači

- Operator `new`
 - Kreira novi objekat
 - Zauzima potrebnu memoriju za čuvanje objekta
 - Objekat se čuva u dijelu memorije koji se naziva *heap*
 - Vraća adresu novokreiranog objekta
 - Omogućava da se u vrijeme izvršavanja zauzima memorija po potrebi
- Operator `delete`
 - Oslobađa memoriju za objekat npr. `delete cPtr;`
 - Objekat za koji se oslobađa memorija mora prethodno biti kreiran putem `new` operatora
- Operator `->`
 - Indirektno (posredno) pristupanje atributima i metodama preko pokazivača

1.6 Reference

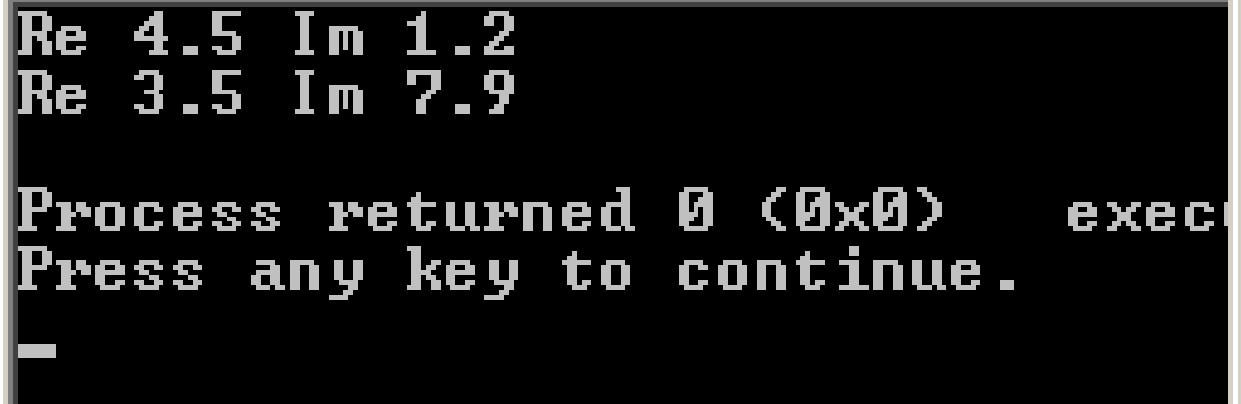
- U pitanju je izvedeni tip
 - Referenca na objekat T je $T\&$
 - Upućuje na objekat T
 - Blizak je pojmu pokazivača
 - U većini slučajeva, realizovana je kao konstantni pokazivač
 - Pokazivač koji je jednom inicijalizovan i ne može se preusmjeriti
 - Trajno upućuje na objekat T
 - Prilikom deklaracije referenca se mora inicijalizovati objektom tipa T
 - Referenca je “čvrsto” vezana za objekat
 - Predstavlja *alias* objekta, tj. njegovo drugo ime
 - Svaka operacija nad referencom je operacija nad objektom

1.6 Reference

- Pristup objektu preko reference je posredan isto kao i kod pokazivača
 - Ipak, ne zahtijeva se upotreba posebnog operatora ($->$) već se koristi operator direktnog pristupa ($.$)
 - Pokazivač se može preusmjeriti, a referenca ne može
- Ne mogu postojati
 - Reference na reference
 - Pokazivači na reference
 - itd.
- U C++, osim po vrijednosti, argument se mogu prenositi i po referenci
 - Najčešće, reference se koriste u tu svrhu

1.6 Reference

```
int main()  
{  
    Complex c1;  
    Complex &c2 = c1;  
    c1.setData(4.5, 1.2);  
    c1.Print();  
    c2.setData(3.5, 7.9);  
    c1.Print();  
    return 0;  
}
```



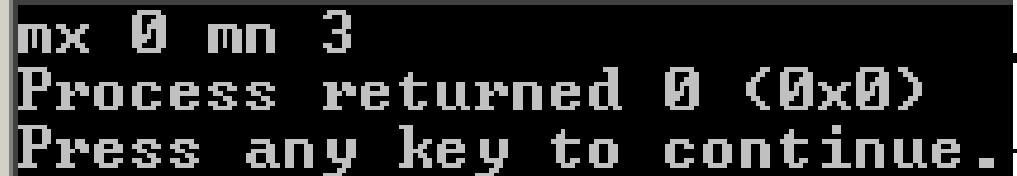
```
Re 4.5 Im 1.2  
Re 3.5 Im 7.9
```

```
Process returned 0 (0x0)   exec  
Press any key to continue.  
_
```

1.6 Reference

```
void maxmin(int a, int b, int mx, int& mn)
{
    if (a < b)
    {
        mx = b;
        mn = a;
    }
    else
    {
        mx = a;
        mn = b;
    }
}
```

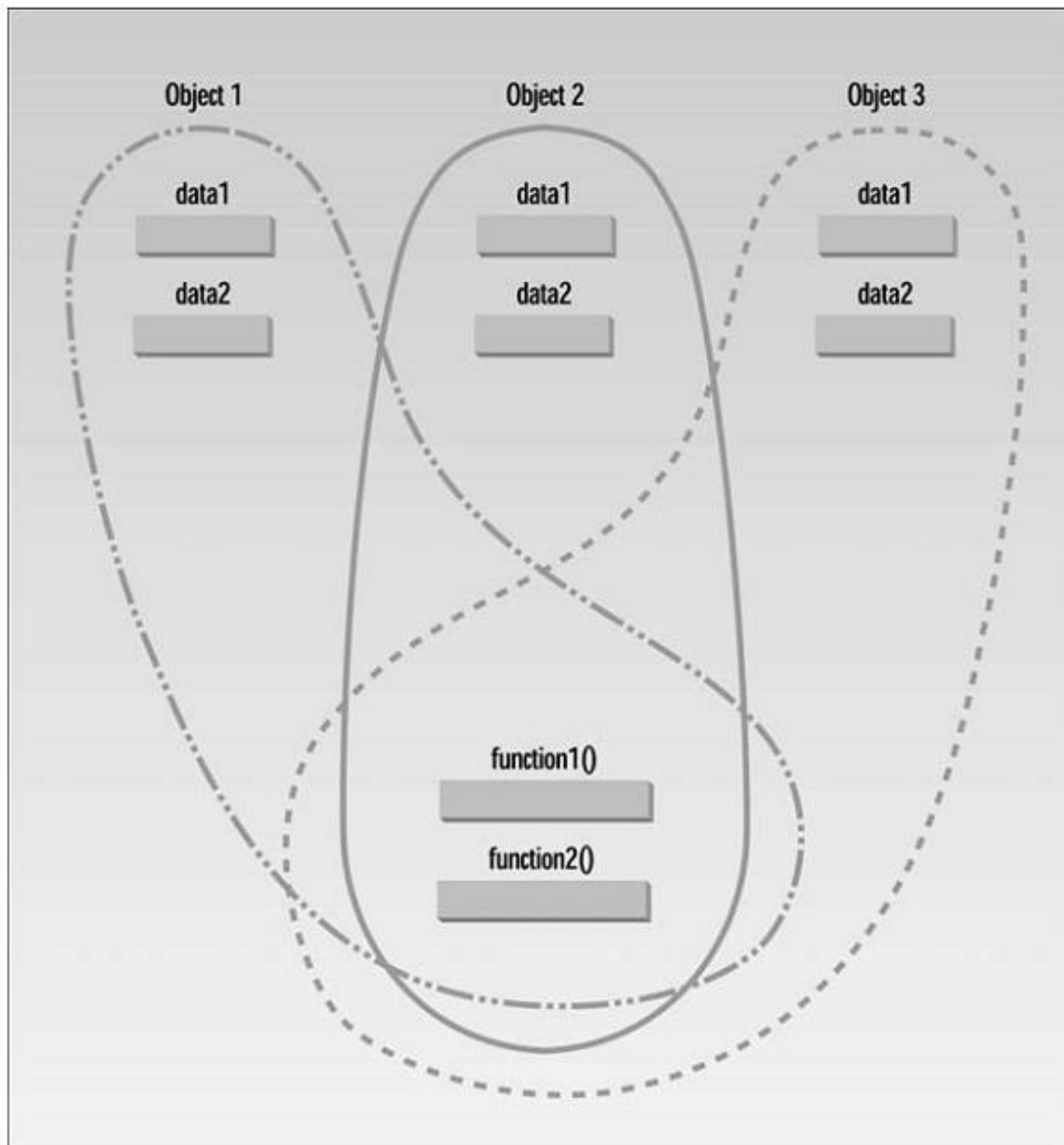
```
int main()
{
    int mx = 0;
    int mn = 0;
    maxmin(3, 5, mx, mn);
    cout << "mx " << mx << " mn " << mn;
}
```



```
mx 0 mn 3
Process returned 0 (0x0)
Press any key to continue.
```

1.7 Objekti i funkcije članice

- Svaki objekat ima svoju kopiju atributa članova
 - Za funkcije članice stvar nije tako jednostavna
- Svi objekti koriste istu implementaciju funkcije
 - Logično, implementacija funkcije je ista za svaki objekat određene klase
- Objekat ne sadrži kopiju funkcije
 - Sadrži adresu funkcije
- Logično pitanje
 - Na osnovu čega funkcija “zna” koji objekat je pozvao njeno izvršavanje



1.8 Pokazivač `this`

- Unutar svake funkcije članice postoji implicitni (ugrađeni) lokalni pokazivač `this`
- Tip ovog pokazivača je "konstantni pokazivač na klasu čija je funkcija članica"
 - Npr. za funkcije članice iz klase `Complex` pokazivač `this` je tipa `Complex* const`
 - Pokazivač je konstantan (`const`), tj. nije dozvoljeno mijenjati vrijednost ovog pokazivača
 - Npr. `this = NULL;` nije validna naredba
- Sadrži adresu objekta putem koga je metoda članica pozvana

1.8 Pokazivač `this`

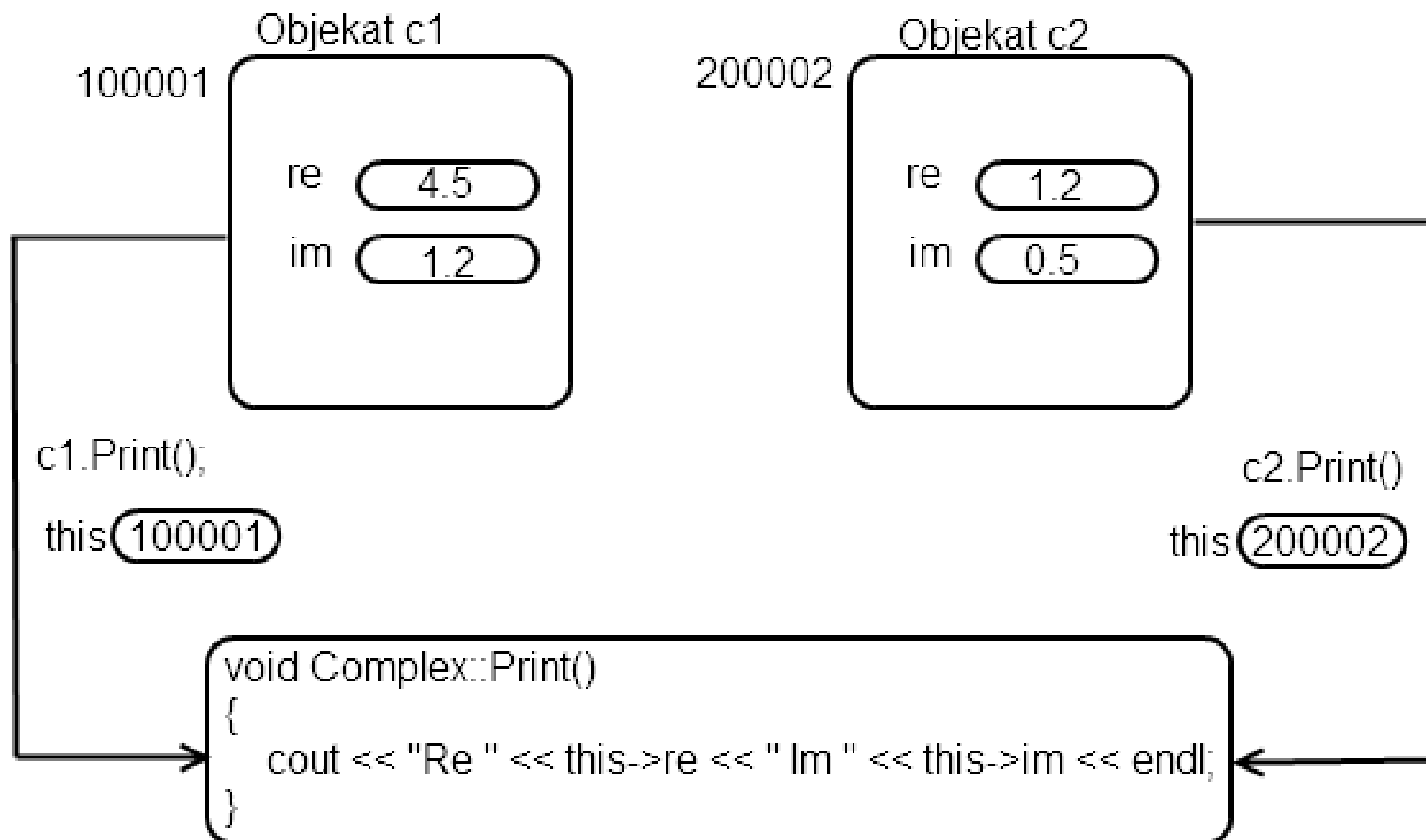
- Implicitno se koristi se za pristup članovima objekta
 - Može se i eksplicitno navesti
- Pokazivač `this` je zapravo skriveni argument metode članice
- Poziv `objekat.func()` je semantički ekvivalentno `f(&objekat)`

1.8 Pokazivač `this`

- Sljedeće implementacije funkcija su ekvivalentne
 - U drugoj je pokazivač `this` eksplicitno naveden

```
void Complex::Print()  
{  
    cout << "Re " << re << " Im " << im <<  
    endl;  
}
```

```
void Complex::Print()  
{  
    cout << "Re " << this->re << " Im " <<  
    this->im << endl;  
}
```



1.9 Preklopljene funkcije

- U klasi je moguće definisati dvije funkcije članice koje imaju isto ime
 - Takve funkcije se nazivaju preklopljene funkcije (*Overloaded Functions*)
 - Moraju da se razlikuju se po broju ili po tipu formalnih argumenata
 - Kada je pozvana preklopljena funkcija, onda se na osnovu poklapanja stvarnih i formalnih argumenata određuje koja verzija funkcije će biti pozvana
 - Postupak se radi u toku prevođenja programa
 - Funkcije koje se razlikuju samo po tipu rezultata ne mogu imati isto ime

1.9 Preklopljene funkcije

```
void Complex::Print()
{
    cout << "Re " << re << " Im " << im << endl;
}

void Complex::Print(int n, char c)
{
    for(int i = 0; i < n; i++)
    {
        cout << c;
    }
    cout << endl << "Re " << re << " Im " << im << endl;
}

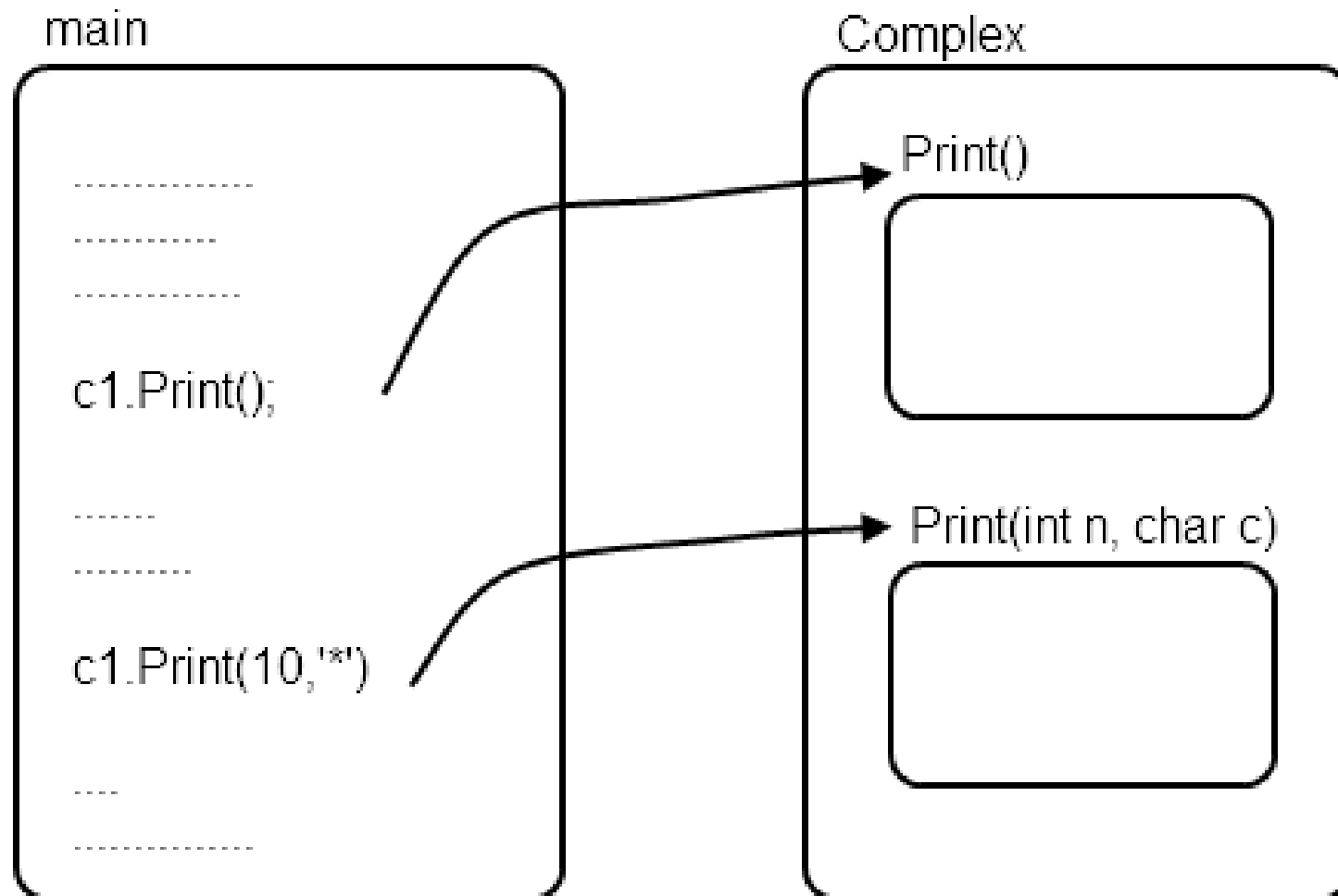
int main()
{
    Complex c1;
    c1.setData(4.5, 1.2);
    c1.Print();
    c1.Print(10, '*');
    return 0;
}
```

Re 4.5 Im 1.2

Re 4.5 Im 1.2

Process returned 0 (0x0) execution time : 0.078
Press any key to continue.

1.9 Preklopljene funkcije



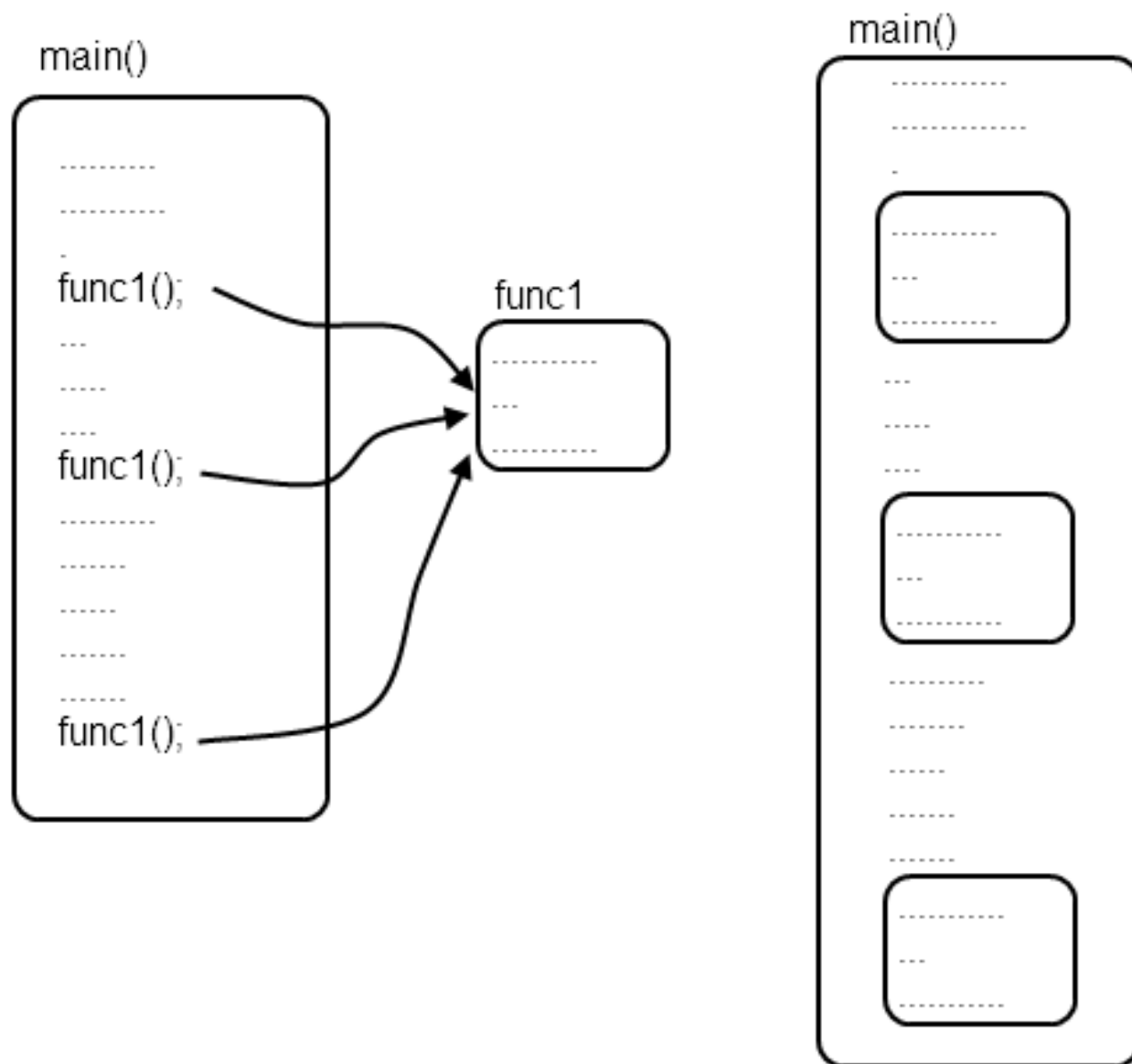
1.10 *Inline* funkcije

- Tijelo funkcije čuva se na jednom mjestu u memoriji
 - Ušteda prostora
 - Kada kompajler naiđe na poziv funkcije, tada se generiše odgovarajuća asemblerska naredba (npr. CALL ili JMP)
 - “Skok” se na prvu naredbu funkcije
 - Obrada poziva funkcije zahtijeva vrijeme
 - Čuva se stanje registara
 - Stvarni argumenti se stavljaju na stek
 - Preuzimanje povratne vrijednosti
 - Restauriranje stanja registara
 - Ovo naročito dolazi do izražaja za kratke funkcije koje imaju nekoliko naredbi
 - Dolazi do situacije da više vremena troši obrada poziva nego samo izvršavanje tijela funkcije

1.10 *Inline* funkcije

- Programski jezik C++ nudi mogućnost neposrednog ugrađivanja tijela funkcije u kod
 - Ovakve funkcije se zovu *inline* funkcije
 - Ispred njihove deklaracije se navodi ključna riječ `inline`
 - Od prevodioca se zahtijeva da optimizuje pozive funkcija time što će umjesto poziva funkcije direktno u kod biti ugrađeno njeno tijelo
 - Ovaj zahtjev prevodilac nije u obavezi da ispuni
 - Semantika programa ostaje ista
 - Ovaj mehanizam treba pažljivo koristiti
 - Izbjegavati za obimne funkcije
 - Pogodno je za funkcije koje imaju par naredbi
 - Funkcije članice su podrazumijevano *inline* ako je njihova implementacija navedena u deklaraciji (interfejsu) klase

1.10 *Inline* funkcije



1.10 *Inline* funkcije

```
class Complex
{
    private:
        float re;
        float im;
    public:
        void setData(float r, float i);
        void Print();
        void Print(int n, char c);
        void setRe(float re){this->re =
re;}
        inline void setIm(float im){this-
>im = im;}
};
```

1.11 Konstante funkcije

- Ove funkcije mogu samo da čitaju stanje objekta ali ne i da mijenjaju njegovo stanje
 - Ne mogu da mijenjaju vrijednosti varijabli
 - Nazivaju se *inspektori* ili *selektori* (engl. *inspector*, *selector*)
 - Za funkcije koje mogu da mijenjaju stanje objekta koristi se termin *mutator* ili *modifikator* (engl. *mutator*, *modifier*)
 - Za deklaraciju ovih funkcija koristi se ključna riječ `const` iza zaglavlja funkcije
 - U pitanju je zapravo dobra programerska praksa
 - Stanje objekta se takođe ne može promijeniti ni putem `this` pokazivača

1.11 Konstante funkcije

```
class Complex
{
    private:
        float re;
        float im;
    public:
        void setData(float r, float i);
        void Print();
        void Print(int n, char c);
        void setRe (float re) const {this->re =
re;}
        inline void setIm(float im){this->im =
im;}
};
```

File	Line	Message
D:\faks\00\New...		In member function 'void Complex::setRe(float) const':
D:\faks\00\New...	16	error: assignment of data-member 'Complex::re' in read-only structure
=== Build finished: 1 errors, 0 warnings ===		

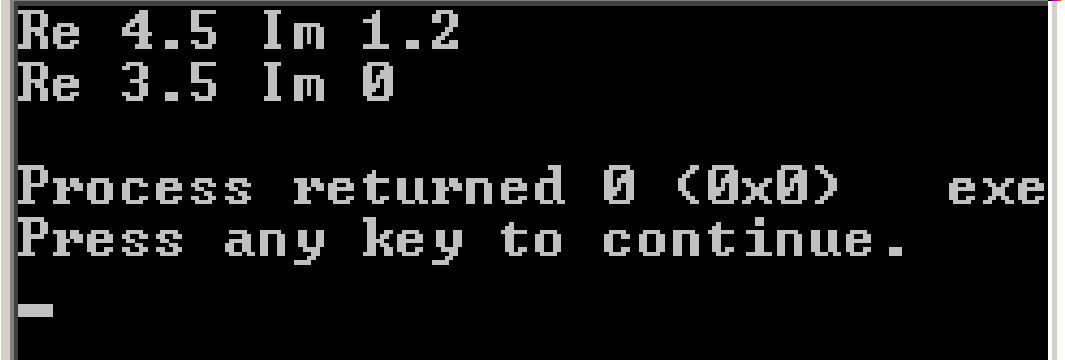
1.12 Podrazumijevani argumenti

- Formalni argument može biti deklarisan tako da ima podrazumijevanu vrijednost
 - Za njih, prilikom poziva funkcije, stvarni argumenti mogu biti izostavljeni
 - Stvarni argument ima podrazumijevanu vrijednost navedenu u deklaraciji
- Podrazumijevanu vrijednost mogu da imaju samo nekoliko posljednjih argumenata iz liste
 - Ako neki argument ima podrazumijevanu vrijednosti svi argumenti navedeni poslije njega u listi moraju imati podrazumijevanu vrijednost
- Podrazumijevana vrijednosti navodi se poslije deklaracije argumenta i to poslije simbola =

1.12 Podrazumijevani argumenti

```
class Complex
{
    private:
        float re;
        float im;
    public:
        void setData(float r, float i = 0.0);
        void Print();
        void Print(int n, char c);
        void setRe (float re) {this->re = re;}
        inline void setIm(float im){this->im = im;}
};

int main()
{
    Complex c1;
    Complex c2;
    c1.setData(4.5, 1.2);
    c2.setData(3.5);
    c1.Print();
    c2.Print();
    return 0;
}
```



```
Re 4.5 Im 1.2
Re 3.5 Im 0
Process returned 0 (0x0)   exe
Press any key to continue.
_
```

1.13 Konstruktori

- U klasi `Complex` postoji metoda `SetData` koja je namijenjena inicijalizaciji članova
 - Ova metoda treba biti pozvana odmah poslije kreiranja objekta
 - Potencijalni izvor grešaka jer ne postoje garancije da će ta metoda biti pozvana
- Potreban je mehanizam koji će omogućiti da se inicijalizacija članica objekta vrši automatski, odmah poslije kreiranja objekta
- Konstruktor:
 - Specifična metoda klase koja definiše početno stanje objekta
 - Nosi isto ime kao i klasa
 - Nema tip rezultata (čak ni `void`)
 - Može imati proizvoljan broj argumenata proizvoljnog tipa

1.13 Konstruktori

- Konstruktor se ne poziva eksplicitno
 - Implicitno prilikom kreiranja objekta
 - Prilikom izvršavanja definicije objekta
 - Prilikom kreiranja putem operatora `new`
 - Kada se stvarni argument klasnog tipa prenosi u formalni (formalni argument se inicijalizuje stvarnim)
- U okviru tijela konstruktora ostalim članovima objekta se pristupa kao i kod drugih funkcija
- Konstruktori mogu biti
 - Preklopljeni
 - Važe ista pravila preklapanja kao i za ostale funkcije
 - *Inline*
 - Javni, privatni ili zaštićenih
 - Važe ista pravila kao i za druge funkcije
 - Ipak, najčešće konstruktori su javni

1.13 Konstruktori

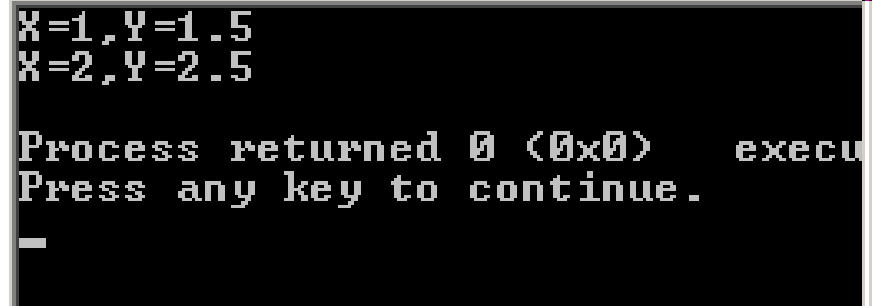
```
class Point
{
    private:
        float x;
        float y;
    public:
        Point(float x, float y);
        void Print();
};

Point::Point(float x, float y)
{
    this->x = x;
    this->y = y;
}

void Point::Print()
{
    cout << "X=" << x << ",Y=" << y << endl;
}
```

1.13 Konstruktori

```
int main()
{
    Point p1(1.0,1.5);
    Point* pPtr = new Point(2.0, 2.5);
    p1.Print();
    pPtr->Print();
    return 0;
}
```



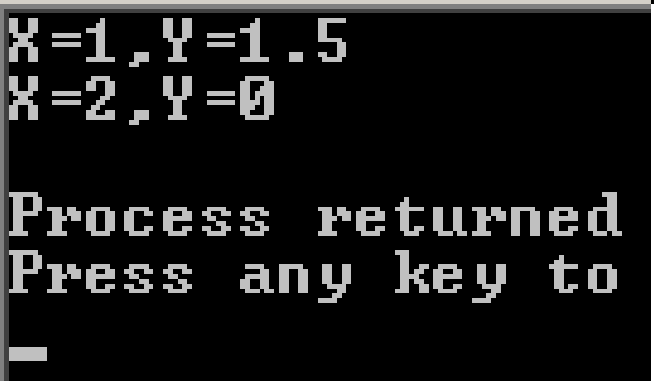
```
X=1,Y=1.5
X=2,Y=2.5
Process returned 0 (0x0)   execu
Press any key to continue.
_
```

- Npr. deklaracija `Point p1(1.0,1.0,1.0);` nema smisla i prevodilac bi prijavio grešku
 - Ne postoji odgovarajući konstruktor

1.13 Konstruktori

```
class Point
{
    private:
        float x;
        float y;
    public:
        Point(float x, float y);
        Point(float x) //inline konstruktor
        {
            this->x = x;
            this->y = 0.0;
        }
        void Print();
};

int main()
{
    Point p1(1.0,1.5);
    Point* pPtr = new Point(2.0);
    p1.Print();
    pPtr->Print();
    return 0;
}
```

A terminal window with a black background and white text. It displays the output of the program: 'X=1,Y=1.5' on the first line and 'X=2,Y=0' on the second line.

X=1,Y=1.5
X=2,Y=0

Process returned
Press any key to
_

1.13 Konstruktori

- Treba razlikovati inicijalizaciju od dodjele
 - U prethodnom primjeru podaci članovi su prostog tipa, pa dodjela i inicijalizacija imaju isto značenje
 - U opštem slučaju, za proizvoljan tip T to ne mora da važi
- Postavlja se pitanje kako inicijalizovati podatke članove u slučaju kada:
 - Kada je član objekat klase za koju ne postoji podrazumijevani konstruktor
 - Kada je član konstanta
 - Kada je član referenca

1.13 Konstruktori

```
class Circle
{
    private:
        Point p;
        float &r;
        const float r2;
    public:
        Circle(float x, float y, float
r) ;
};
```

1.13 Konstruktori

- Prije izvršavanja konstruktora pozivaju se konstruktori članova
 - Redoslijedom pojavljivanja u klasi
- Inicijalizatori (argumenti ovih poziva) mogu da se navedu iza zaglavlja definicije (ne deklaracije) konstruktora, iza znaka :
 - Tzv. inicijalizatori članova
 - Odvojeni su zarezom
 - Mogu se koristiti argumenti konstruktora, kao i konstante
 - Poželjno je da se navedu u redoslijedu u kojem su deklarirani u klasi

1.13 Konstruktori

```
class Circle
{
    private:
        Point p;
        float r;
        const float PI;
    public:
        Circle(float x, float y, float r)
        : p(x,y), r(r), PI(3.14)
        {
        }
};
```


1.13 Konstruktori

```
class Complex
{
    private:
        float re;
        float im;
    public:
        Complex(float r, float i);
};

Complex::Complex(float r, float i) :
    re(r), im(i) {};
```

1.13 Konstruktori

- Podrazumijevani konstruktor
 - Nema formalnih argumenata
 - Ili svi argumenti imaju podrazumijevanu vrijednost
 - Poziva se bez stvarnih argumenata
 - Ugrađeni podrazumijevani konstruktor
 - Nema formalnih argumenata
 - Ima prazno tijelo
 - Postoji samo ako klasa nije definisala ni jedan konstruktor
 - Time se obezbjeđuje da svaka klasa ima makar jedan konstruktor

1.13 Konstruktori

```
class Point
{
    private:
        float x;
        float y;
    public:
        void Print();
};

int main()
{
    //poziv ugrađenog podrazumijevanog konstruktora
    Point* pPtr = new Point();
    pPtr->Print();
    return 0;
}
```

1.13 Konstruktori

```
class Point
{
    private:
        float x;
        float y;
    public:
        Point(float x, float y);
        void Print();
};

int main()
{
    //Greska. Ugrađeni podrazumijevani konstruktor vise ne
    postoji
    Point* pPtr = new Point();
    pPtr->Print();
    return 0;
}
```

1.13 Konstruktori

```
class Point
{
    private:
        float x;
        float y;
    public:
        Point(float x, float y) : x(x), y(y) {}
        Point(float x) : x(x), y(0.0) {}
        Point() : x(0.0), y(0.0) {}
        void Print();
};

int main()
{
    //Validno
    Point* pPtr = new Point();
    pPtr->Print();
    return 0;
}
```

1.13 Konstruktori

- Kada se jedan objekat t_1 klase T inicijalizuje drugim objektom t_2 iste te klase, onda će se izvršiti prosta inicijalizacija članova objekta t_1 odgovarajućim članovima objekta t_2
 - Ovo može biti problematično ako je pokazivač član klase
 - Dolazi do tzv. plitkog dodjeljivanja
 - Ako pokazivač ukazuje na objekat koji je kreiran dinamički, onda će oba objekta imati pokazivač na taj isti objekat koji je kreiran dinamički

1.13 Konstruktori

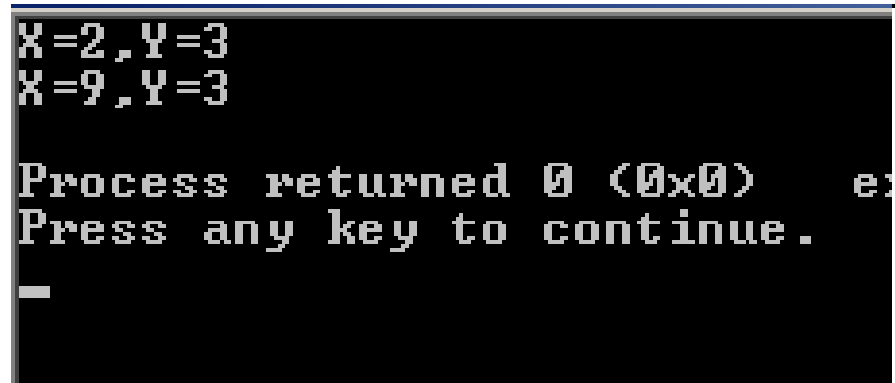
```
class Point
{
    private:
        float x;
        float y;
    public:
        Point(float x = 0.0, float y =
0.0) : x(x), y(y) {}
        float getX() {return this->x;}
        float getY() {return this->y;}
        void setX(float x) {this->x = x;}
        void setY(float y) {this->y = y;}
        void Print();
};
```

1.13 Konstruktori

```
class Circle
{
    private:
        Point* p;
        float r;
        const float PI;
    public:
        Circle(float x, float y, float r)
: r(r), PI(3.14)
        {
            p = new Point(x, y);
        }
        Point* getP() {return p;}
};
```


1.13 Konstruktori

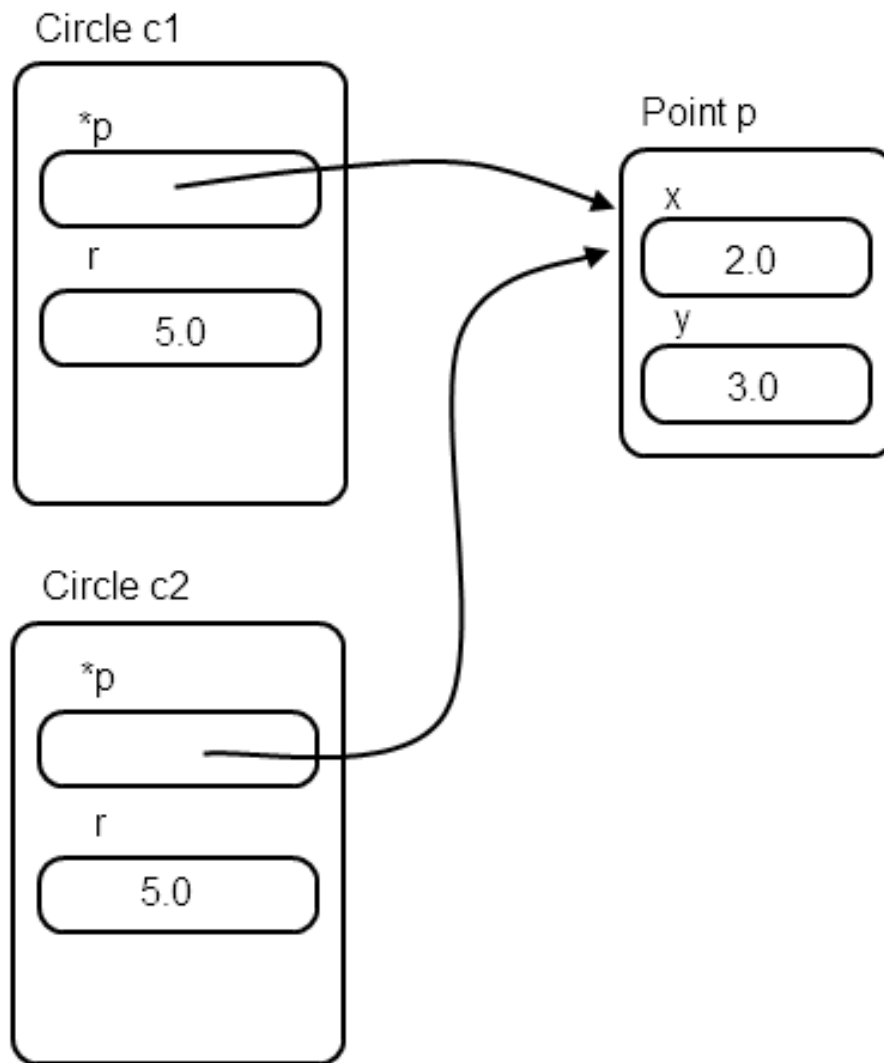
```
int main()
{
    Circle c1(2.0, 3.0, 5.0);
    // poziva se ugradjeni konstruktor kopije
    Circle c2 = c1;
    c1.getP()->Print();
    c2.getP()->setX(9.0);
    c1.getP()->Print();
    return 0;
}
```



```
X=2,Y=3
X=9,Y=3
```

```
Process returned 0 (0x0)
Press any key to continue.
```

1.13 Konstruktori



1.13 Konstruktori

- Potpuna kontrola nad inicijalizacijom jednog objekta drugim može se ostvariti definisanjem tzv. konstruktora kopije
- Konstruktor kopije poziva se kada se objekat inicijalizuje objektom iste klase, a to je:
 - Prilikom inicijalizacije objekta (pomoću znaka = ili sa zagradama)
 - Prilikom prenosa argumenata u funkciju (kreira se lokalni objekat)
 - Prilikom vraćanja vrijednosti iz funkcije (kreira se privremeni objekat)
- Konstruktor kopije ima argument tipa `XX&` ili `const XX&`
 - Dodatno, može imati još argmenata, ali oni moraju imati definisanu podrazumijevanu vrijednost
- Konstruktor ne smije imati formalni argument tipa svoje klase
 - Beskonačna rekurzija

1.13 Konstruktori

```

class Circle
{
    private:
        Point* p;
        float r;
        const float PI;
    public:
        Circle(Circle &x); //konstruktor kopije
        .....
};

Circle::Circle(Circle &x) : PI(3.14)
{
    cout << "konstruktor kopije";
    this->r = x.r;
    this->p = new Point();
    this->p->setX(x.p->getX());
    this->p->setY(x.p->getY());
}

```

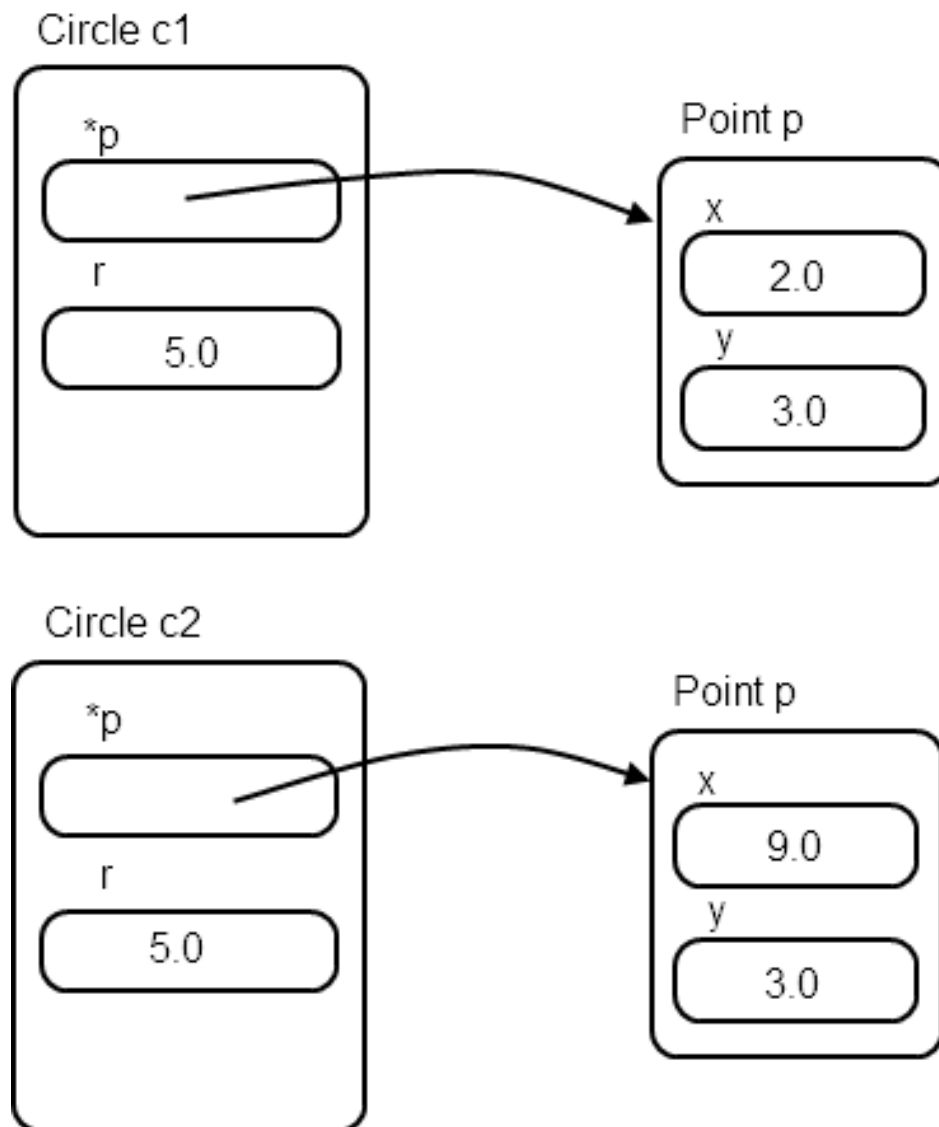
1.13 Konstruktori

```
int main()
{
    Circle c1(2.0, 3.0, 5.0);
    // poziva se predefinisani konstruktor
    kopije
    Circle c2 = c1;
    c1.getP()->Print();
    c2.getP()->setX(9.0);
    c1.getP()->Print();
    return 0;
}
```

```
konstruktor kopije
X=2,Y=3
X=2,Y=3
```

```
Process returned 0 (0x0)   a
Press any key to continue.
```

1.13 Konstruktori



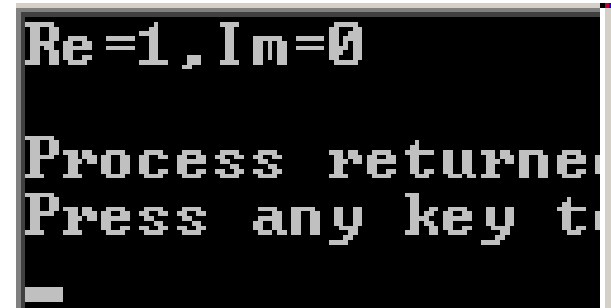
1.13 Konstruktori

- Korisnička konverzija (engl. *user-defined conversion*)
 - Ako klasa X ima konstruktor koji se može pozvati sa jednim stvarnim argumentom tipa T , ovim konstruktorom se definiše korisnička konverzija iz tipa T u tip X
 - Tip T može biti ugrađni ili korisnički definisan tip
 - Implicitno se koristi tamo gdje je potrebna
 - Pri inicijalizaciji objekta tipa X objektom tipa T

1.13 Konstruktori

```
class Complex
{
    private:
        float re;
        float im;
    public:
        Complex(float r = 0.0, float i = 0.0) : re(r),
        im(i) {};
        void Print(){cout << "Re=" << re << ",Im=" << im <<
        endl;}
};
```

```
int main()
{
    Complex c1 = 1.0;
    //Dozvoljeno. Isto kao
    //da je napisano
    //Complex c1(1.0);
    c1.Print();
    return 0;
}
```



Re=1,Im=0

Process returned
Press any key to

1.13 Konstruktori

- Privremeni objekti
 - Bezimeni
 - Programer nema kontrolu nad njima
 - U potpunosti pod kontrolom prevodica
 - Ako se na nekom mjestu uvodi privremeni objekat koji ima konstruktor, onda se obavezno i poziva taj konstruktor
 - Slično važi i za destruktore
 - Prevodilac određuje kada će neki privremeni objekat biti kreiran, a kada uništen
 - Vrše se optimizacija
 - Semantika programa ostaje sačuvana

1.13 Konstruktori

```
class Complex
{
    public:
        float re;
        float im;
    public:
        Complex(float r = 0.0, float i = 0.0) : re(r),
        im(i) {};
        void Print(){cout << "Re=" << re << ",Im=" << im <<
        endl;}
};
```

```
Complex add(Complex c1, Complex c2)
{
    return Complex(c1.re + c2.re, c1.im + c2.im);
}
int main()
{
    Complex c1;
    c1 = add(Complex(1.0,2.0), Complex(4.5, 6.5));
    c1.Print();
    return 0;
}
```

Re=5.5,Im=8.5

Process returned 0 (0x
Press any key to conti

1.14 Destruktori

- Destruktor:
 - Specifična funkcija članica klase koja se implicitno poziva pri uništavanju objekta, tj. na kraju životnog vijeka
 - Nosi isto ime kao klasa, uz znak ~ ispred imena
 - Nema tip rezultata i ne može imati argumente
 - Klasa može imati samo jedan destruktor
 - U okviru destruktora članovima se pristupa kao i u okviru drugih funkcija
 - Poziva se i pri uništavanju dinamičkog objekta pomoću operatora `delete`
 - Destruktori se dosta koriste
 - Kada objekat treba da dealocira memoriju koju je konstruktor alocirao

1.14 Destruktori

```
class Point
{
    private:
        float x;
        float y;
    public:
        Point(float x = 0.0, float y = 0.0) :
        x(x), y(y) {cout << "Point : konstruktor" <<
        endl;}
        Point(Point& p) : x(p.x), y(p.y) {cout <<
        "Point : konstruktor kopije" << endl;}
        ~Point() {cout << "Point : destruktor" <<
        endl;}
        float getX() {return this->x;}
        float getY() {return this->y;}
        void setX(float x) {this->x = x;}
        void setY(float y) {this->y = y;}
        void Print();
};
```

1.14 Destruktori

```
class Circle
{
    private:
        Point* p;
        float r;
        const float PI;
    public:
        Circle(Circle &x);
        Circle(float x, float y, float r) : r(r), PI(3.14)
        {
            cout << "Circle : konstruktor" << endl;
            p = new Point(x, y);
        }
        ~Circle();
        Point* getP(){return p;}
};

Circle::~~Circle()
{
    cout << "Circle : destructor " << endl;
    delete p;
}
```

1.14 Destruktori

```
int main()
{
    Circle* c1 = new Circle(2.0, 3.0,
    5.0);
    delete c1;
    return 0;
}
```

```
Circle : konstruktor
Point : konstruktor
Circle : destructor
Point : destruktor
```

```
Process returned 0 (0x0)   execution t
Press any key to continue.
```

1.15 Objekti kao argumenti funkcija

- Do sada argumenti funkcija su bili prostog tipa (*int*, *float* itd.)
 - Argument funkcije može da bude i objekat
 - Nema ograničenja, tj. može biti objekat bilo koje klase
 - Takođe, argument funkcije može da pokazivač, kao i referenca
- Argument funkcije je lokalni objekat
 - Životni vijek objekta ograničen je trajanjem izvršavanja funkcije
 - Poziva se konstruktor kopije
 - Kopira se objekat koji se stvarni argument
 - U slučaju da konstruktor kopije nije definisan od strane korisnika, poziva se podrazumijevani konstruktor kopije

1.15 Objekti kao argumenti funkcija

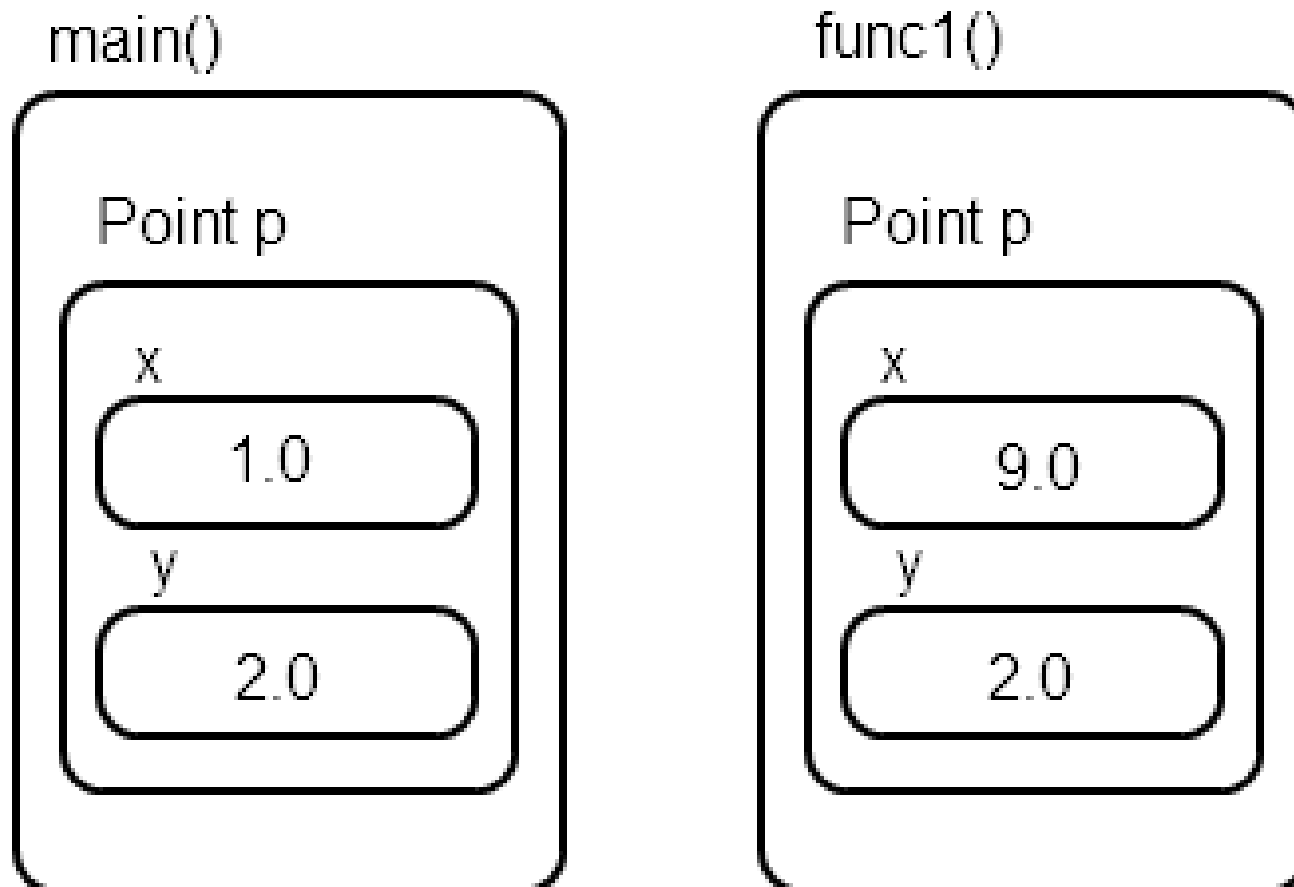
```
void func1(Point p)
{
    p.setX(9.0)
}

int main()
{
    Point p(1.0, 2.0);
    p.Print();
    func1(p);
    p.Print();
    return 0;
}
```

```
Point : konstruktor
X=1,Y=2
Point : konstruktor kopije
Point : destruktor
X=1,Y=2
Point : destruktor

Process returned 0 (0x0)   e:
Press any key to continue.
```


1.15 Objekti kao argumenti funkcija



1.15 Objekti kao argumenti funkcija

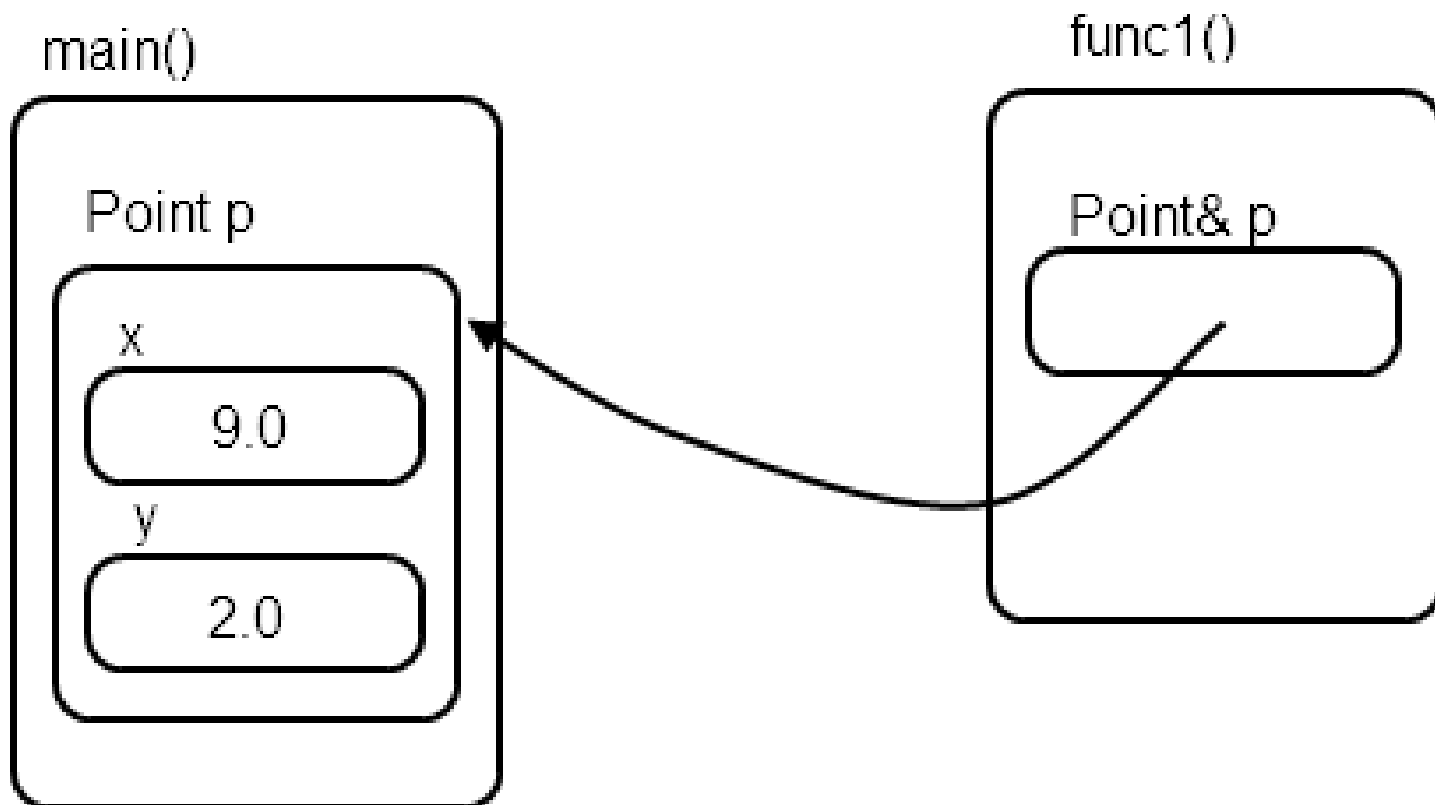
```
void func1(Point& p)
{
    p.setX(9.0);
}
```

```
int main()
{
    Point p(1.0, 2.0);
    p.Print();
    func1(p);
    p.Print();
    return 0;
}
```

```
Point : konstruktor
X=1,Y=2
X=9,Y=2
Point : destruktor
```

```
Process returned 0 (0x0)   execution time
Press any key to continue.
```

1.15 Objekti kao argumenti funkcija



1.16 Statički (dijeljeni) atributi

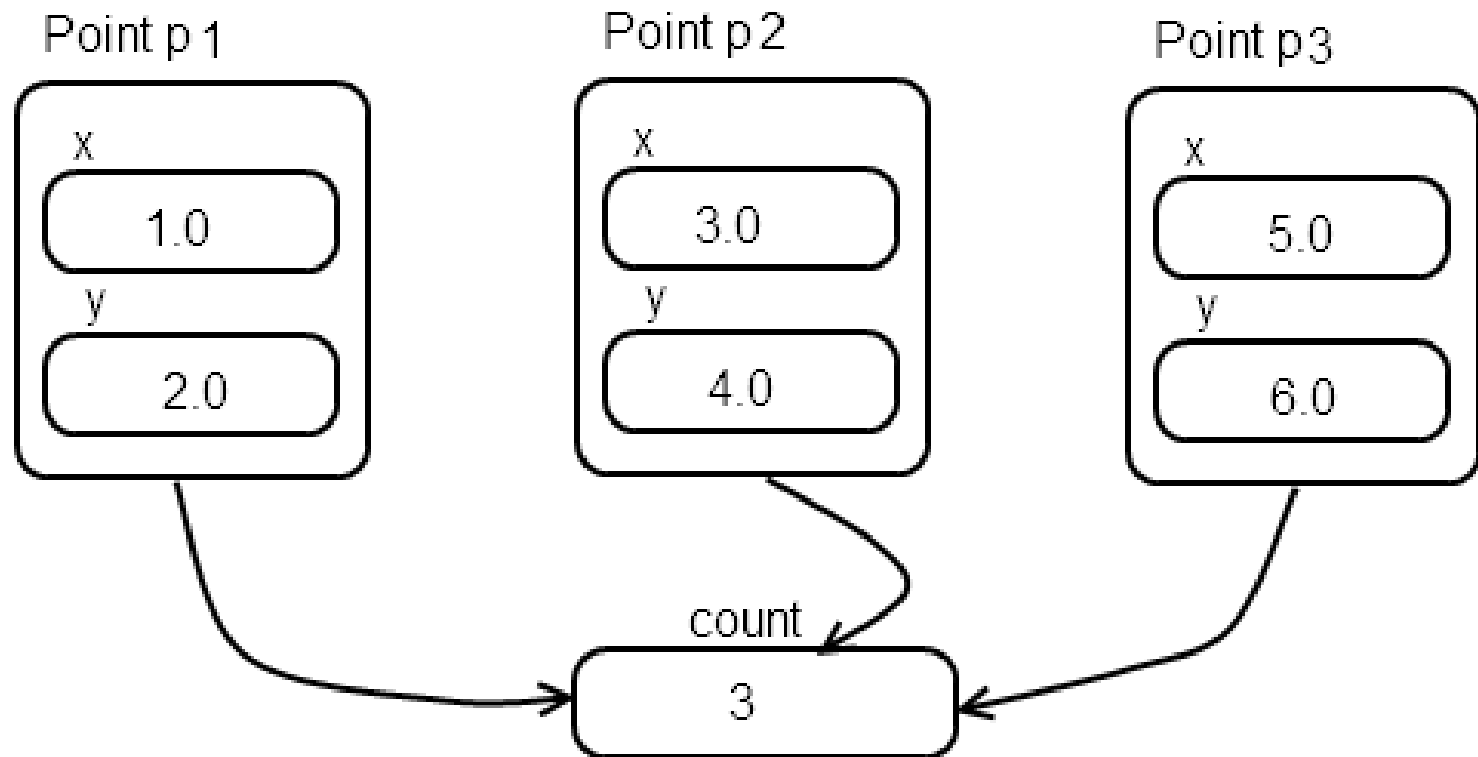
- Svaki objekat klase posjeduje poseban komplet atributa te klase
 - Ipak, moguće je deklarirati attribute koji su zajednički za sve objekte
- Atributi koji su zajednički za sve objekte nazivaju se statičkim atributima, tj. statičkim podacima članovima
 - Deklarišu se navođenjem ključne riječi `static`
- Postoji samo jedan primjerak statičkog atributa u memoriji za cijelu klasu
 - zajednički podatak za sve objekte klase

```

class Point
{
    private:
        float x;
        float y;
    public:
        static int count;
    public:
        Point(float x = 0.0, float y = 0.0) : x(x), y(y)
        {
            count = count + 1;
            cout << "Point : konstruktor " << endl;
        }
        Point(Point& p) : x(p.x), y(p.y)
        {
            count = count + 1;
            cout << "Point : konstruktor kopije" << endl;
        }
        ~Point()
        {
            count = count - 1;
            cout << "Point : destruktork" << endl;
        }
        float getX(){return this->x;}
        float getY(){return this->y;}
        void setX(float x){this->x = x;}
        void setY(float y){this->y = y;}
        void Print();
};

```

1.16 Statički (dijeljeni) atributi



1.16 Statički (dijeljeni) atributi

- Svaki pristup statičkom atributu iz bilo kojeg objekta znači i pristup istoj oblasti u memoriji
- Statički atributi postoje tokom cijelog izvršavanja programa
 - Ne zavisi od postojanja bilo kojeg objekta klase
 - Postoji i kada ne postoji ni jedan objekat klase
- Deklaracija statičkog člana nije i njegova definicija
 - Mora se definisati posebnom deklaracijom
 - Izvan deklaracije klase
 - Izvan deklaracije funkcija
 - Npr. `int Point::count = 0;`

1.16 Statički (dijeljeni) atributi

- Statičkim atributima može se pristupiti bez navođenja konkretnog objekta
 - Prije imena atributna navodi operator `::` i ime klase
 - Ako je `count` statički atribut klase `Point`, onda mu se pristupa sa `Point::count`
- Statičkim atributima može se pristupiti i preko operator pristupa (`.` i `->`)
 - Važe ista pravila kao i za nestatičke članove
- Kontrola pristupa važi kao i za ostale članove
- Koriste se kada svi primjerci jedne klase dijele zajedničku informaciju
 - Osobina klase, a ne konkretnog atributa

1.16 Statički (dijeljeni) atributi

```
int Point::count = 0;

int main()
{
    Point* p1 = new Point(1.0, 2.0);
    cout << "Broj objekata klase point " << p1->count << endl;

    Point* p2 = new Point(3.0, 4.0);
    cout << "Broj objekata klase point " << p2->count << endl;

    Point* p3 = new Point(5.0, 6.0);
    cout << "Broj objekata klase point " << p3->count << endl;

    delete p1;
    cout << "Broj objekata klase point " << Point::count << endl;

    delete p2;
    cout << "Broj objekata klase point " << Point::count << endl;

    delete p3;
    cout << "Broj objekata klase point " << Point::count << endl;
    return 0;
}
```

1.16 Statički (dijeljeni) atributi

```
Point : konstruktor  
Broj objekata klase point 1  
Point : konstruktor  
Broj objekata klase point 2  
Point : konstruktor  
Broj objekata klase point 3  
Point : destruktor  
Broj objekata klase point 2  
Point : destruktor  
Broj objekata klase point 1  
Point : destruktor  
Broj objekata klase point 0  
  
Process returned 0 (0x0)    execution time :  
Press any key to continue.  
_
```

1.13 Statičke (zajedničke) metode

- Metode, kao i atributi, mogu da budu deklarisanе kao statičke (zajedničke) za cijelu klasu
 - Nazivaju se *statičkim metodama članicama*
 - Navodi se ključna riječ `static` ispred deklaracije funkcije
 - Statičke metode ne pripadaju ni jednom objektu
 - Može im se pristupiti iako nije kreiran nijedan objekat klase
 - Ne posjeduju pokazivač `this`
 - Ne mogu pristupi nestatičkim atributima klase kojoj pripadaju
 - Pristup statičkim metodama je sličan kao i pristup statičkim atributima
 - Navodi se ime klase i operator `::` ispred imena funkcije
 - Npr. `Point::getCount()`
 - Može se pristupiti i preko operator pristupa `(. i ->)`
 - Važe ista pravila kao i za druge metode

```
class Point
{
    private:
        float x;
        float y;
        static int count;
    public:
        Point(float x = 0.0, float y = 0.0) : x(x), y(y)
        {
            count = count + 1;
            cout << "Point : konstruktor " << endl;
        }
        Point(Point& p) : x(p.x), y(p.y)
        {
            count = count + 1;
            cout << "Point : konstruktor kopije" << endl;
        }
        ~Point()
        {
            count = count - 1;
            cout << "Point : destruktor" << endl;
        }
        float getX(){return this->x;}
        float getY(){return this->y;}
        void setX(float x){this->x = x;}
        void setY(float y){this->y = y;}
        void Print();
        static int getCount(){return count;}
};
```

1.17 Statičke (zajedničke) metode

```
int Point::count = 0;

int main()
{
    Point* p1 = new Point(1.0, 2.0);
    cout << "Broj objekata klase point " << p1->getCount() << endl;

    Point* p2 = new Point(3.0, 4.0);
    cout << "Broj objekata klase point " << p2->getCount() << endl;

    Point* p3 = new Point(5.0, 6.0);
    cout << "Broj objekata klase point " << p3->getCount() << endl;

    delete p1;
    cout << "Broj objekata klase point " << Point::getCount() << endl;

    delete p2;
    cout << "Broj objekata klase point " << Point::getCount() << endl;

    delete p3;
    cout << "Broj objekata klase point " << Point::getCount() << endl;
    return 0;
}
```

1.18 Prijatelji klasa

- U nekim slučajevima potrebno je da klasa ima “povlašćene” korisnike koji mogu da pristupaju privatnim i zaštićenim članicama klase
 - Prijatelji klase
- Prijatelji klase mogu biti
 - Funkcije
 - Klase
- Prijateljstvo je relacija koja reguliše pravo pristupa
- Prijateljstvo:
 - Se ne nasljeđuje
 - Nije simetrična relacija
 - Nije tranzitivna relacija

1.18 Prijatelji klasa

- Prijateljske funkcije su funkcije koje nisu članice klase, ali imaju dozvoljen pristup do privatnih članova klase
- Prijateljske funkcije mogu da budu:
 - Globalne funkcije ili
 - Članice drugih klasa
- Funkcija je prijateljska:
 - Ako se u definiciji klase navede deklaracija funkcije sa ključnom reči `friend`
- Nevažno je pravo pristupa sekciji klase (privatno, zaštišeno, javno) u kojoj se navodi deklaracija prijateljske funkcije

1.18 Prijatelji klasa

- Funkcija može da bude prijatelj većem broju klasa istovremeno
- Ne narušava se princip enkapsulacije
 - Projektant klase sam određuje koje funkcije će biti prijateljske
 - Izbjegava se “publikovanje” članova svim ostalim klasama
- Globalne prijateljske funkcije najčešće imaju jedan ili više argumenata koji su tipa te klase
 - Posebne usluge koje su vezane za klasu, a ne za objekat
 - *Utilities*
- Pogodnije za korišćenje
 - Kada funkcija treba da pristupa članovima više klasa, efikasnija je prijateljska globalna funkcija
 - Kada se preklapaju operatori, često je jednostavnije definisati globalne (operatorske) funkcije nego metode

1.18 Prijateljli klasa

```
class Complex
{
    private:
        float re;
        float im;
    public:
        Complex(float r, float i) : re(r),
im(i) {};
        void Print();
        friend int IsEqual(Complex* c1, Complex*
c2);
};
```

1.18 Prijatelji klasa

```
int IsEqual(Complex* c1, Complex* c2)
{
    if (c1->re == c2->re && c1->im == c2->im)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}

int main()
{
    Complex* c1 = new Complex(1.0,2.0);
    Complex* c2 = new Complex(1.0,2.0);

    IsEqual(c1, c2);

    delete c1;
    delete c2;
    return 0;
}
```

1.18 Prijatelji klasa

```
class B;

class A {
public:
    int Func1( B& b );

private:
    int Func2( B& b );
};

class B {
private:
    int _b;
    friend int A::Func1( B& );
};

int A::Func1( B& b ) { return b._b; }    // OK
int A::Func2( B& b ) { return b._b; }    // Greska
```

1.18 Prijatelji klasa

- Ako su sve metode klase Y prijateljske funkcije klasi X , onda je Y prijateljska klasa (*friend class*) klasi X
- Bilo koja metoda klase Y može da pristupa svim članovima klase X
- Ovaj princip se koristi kada dvije klase tijesno sarađuju
 - Npr. može se obezbijediti da samo klasa Y kreira objekte klase X
 - Postiže se tako što se konstruktori klase X deklarišu kao privatni, a zatim se klasa Y deklariše kao prijateljska klasi X

1.18 Prijateljli klasa

```
class Point
{
    private:
        float x;
        float y;
        static int count;
    public:
        Point(float x = 0.0, float y = 0.0);
        Point(Point& p) : x(p.x), y(p.y);
        ~Point();
        void Print();
        static int getCount(){return count;}
        friend class Circle;
};
```

1.18 Prijatelji klasa

```
Circle::Circle(Circle &x) : PI(3.14)
{
    cout << "konstruktor kopije" << endl;
    this->r = x.r;
    this->p = new Point();
    this->p->x = x.p->x;
    this->p->y = x.p->y;
}
```